

Electronics and Computer Science  
School of Computer Science  
University of Southampton Malaysia

Abdullah Hashmat Azeez

4 May 2026

Phishing Detection and Awareness Tool

Project supervisor: Dr Syed Hamid Hussain Madni  
Second examiner: Dr Rusyaizila Ramli

A project report submitted for the award of  
BSc Computer Science

## Abstract

Phishing remains the most prevalent cyberattack vector, succeeding not through technical exploitation but through manipulation of human judgement. Existing defences address detection and user education in isolation, and the publicly available email corpora that support the detection literature conflate commercial spam with genuine phishing. This project presents a dual-function web application integrating a machine-learning-based phishing detection engine with an interactive educational platform inside an accessible interface. The detection engine uses a Random Forest classifier trained on 322 features (22 engineered and 300 TF-IDF) extracted from a methodologically audited corpus of 27,180 emails drawn from five public sources, achieving a 97.18% F1-score and 0.9976 ROC-AUC on a held-out test partition. A rule-based explanation engine translates the classifier's output into human-readable threat flags and recommendations. The educational platform delivers sixteen simulation scenarios across three difficulty tiers with contextual feedback and session-based progress tracking, passing a heuristic evaluation against Nielsen's ten usability principles. Expert evaluation confirms that the integrated design strengthens user awareness by connecting detection outputs to educational content, enabling users to learn from analysis rather than instruction. Per-source evaluation exposes a 5.78-percentage-point recall gap between advance-fee fraud and credential phishing, demonstrating that aggregate metrics alone are insufficient for transparent classifier reporting.

## Statement of Originality

- I have read and understood the [ECS Academic Integrity](#) information and the University's [Academic Integrity Guidance for Students](#).
- I am aware that failure to act in accordance with the [Regulations Governing Academic Integrity](#) may lead to the imposition of penalties which, for the most serious cases, may include termination of programme.
- I consent to the University copying and distributing any or all of my work in any form and using third parties (who may be based outside the EU/EEA) to verify whether my work contains plagiarised material, and for quality assurance purposes.

***You must change the statements in the boxes if you do not agree with them.***

We expect you to acknowledge all sources of information (e.g. ideas, algorithms, data) using citations. You must also put quotation marks around any sections of text that you have copied without paraphrasing. If any figures or tables have been taken or modified from another source, you must explain this in the caption and cite the original source.

**I have acknowledged all sources, and identified any content taken from elsewhere.**

If you have used any code (e.g. open-source code), reference designs, or similar resources that have been produced by anyone else, you must list them in the box below. In the report, you must explain what was used and how it relates to the work you have done.

**I have not used any resources produced by anyone else.**

You can consult with module teaching staff/demonstrators, but you should not show anyone else your work (this includes uploading your work to publicly-accessible repositories e.g. Github, unless expressly permitted by the module leader), or help them to do theirs. For individual assignments, we expect you to work on your own. For group assignments, we expect that you work only with your allocated group. You must get permission in writing from the module teaching staff before you seek outside assistance, e.g. a proofreading service, and declare it here.

**I did all the work myself, or with my allocated group, and have not helped anyone else.**

We expect that you have not fabricated, modified or distorted any data, evidence, references, experimental results, or other material used or presented in the report. You must clearly describe your experiments and how the results were obtained, and include all data, source code and/or designs (either in the report or submitted as a separate file) so that your results could be reproduced.

**The material in the report is genuine, and I have included all my data/code/designs.**

We expect that you have not previously submitted any part of this work for another assessment. You must get permission in writing from the module teaching staff before re-using any of your previously submitted work for this assessment.

**I have not submitted any part of this work for another assessment.**

If your work involved research/studies (including surveys) on human participants, their cells or data, or on animals, you must have been granted ethical approval before the work was carried out, and any experiments must have followed these requirements. You must give details of this in the report and list the ethical approval reference number(s) in the box below.

**My work did not involve human participants, their cells or data, or animals.**

## **Acknowledgements**

I would like to thank my project supervisor, Dr. Madni, for his consistent guidance, constructive feedback, and encouragement throughout the duration of this project. His weekly supervision sessions were instrumental in shaping both the technical direction and the academic integrity of the work.

I would also like to thank my project examiner, Dr. Zila, who was very supportive throughout the project and provided helpful advice when it was needed. I extend my appreciation to the academic staff who provided informal feedback on the educational platform during the evaluation phase.

Furthermore, I am grateful to the University of Southampton's School of Electronics and Computer Science for providing the resources and infrastructure that made this project possible.

Finally, I would like to acknowledge my family and friends for their fierce and loyal support throughout my journey.

## Table of Contents

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| <b>1</b> | <b>CHAPTER 1: INTRODUCTION .....</b>                                | <b>1</b>  |
| 1.1      | PROBLEM STATEMENT .....                                             | 1         |
| 1.2      | PROJECT AIMS AND OBJECTIVES .....                                   | 1         |
| 1.3      | CONTRIBUTIONS .....                                                 | 2         |
| 1.4      | SCOPE AND LIMITATIONS .....                                         | 2         |
| 1.5      | PROJECT SIGNIFICANCE .....                                          | 3         |
| 1.6      | REPORT STRUCTURE .....                                              | 3         |
| <b>2</b> | <b>CHAPTER 2: LITERATURE REVIEW .....</b>                           | <b>4</b>  |
| 2.1      | THE PHISHING THREAT LANDSCAPE .....                                 | 4         |
| 2.2      | TECHNICAL APPROACHES TO PHISHING DETECTION .....                    | 4         |
| 2.3      | SECURITY AWARENESS TRAINING .....                                   | 6         |
| 2.4      | PHISHING CHARACTERISATION AND EXPLANATION .....                     | 7         |
| 2.5      | FINDINGS FROM THE LITERATURE REVIEW .....                           | 8         |
| <b>3</b> | <b>CHAPTER 3: PROPOSED METHODOLOGY .....</b>                        | <b>9</b>  |
| 3.1      | METHODOLOGY FRAMEWORK .....                                         | 9         |
| 3.2      | DATASETS .....                                                      | 9         |
| 3.3      | FEATURE ENGINEERING .....                                           | 10        |
| 3.4      | DATA PARTITIONING AND CLASS IMBALANCE .....                         | 13        |
| 3.5      | MODEL SELECTION AND HYPERPARAMETER TUNING .....                     | 14        |
| 3.6      | TRAINING PIPELINE .....                                             | 14        |
| 3.7      | ARTEFACT EXPORT .....                                               | 16        |
| <b>4</b> | <b>CHAPTER 4: ARCHITECTURE AND TECHNICAL IMPLEMENTATION .....</b>   | <b>17</b> |
| 4.1      | ARCHITECTURAL OVERVIEW .....                                        | 17        |
| 4.2      | BACKEND SERVICES .....                                              | 17        |
| 4.3      | INFERENCE PIPELINE .....                                            | 19        |
| 4.4      | EXPLANATION ENGINE .....                                            | 20        |
| 4.5      | EDUCATIONAL PLATFORM .....                                          | 21        |
| 4.6      | FRONTEND ARCHITECTURE .....                                         | 23        |
| 4.7      | DATABASE SCHEMA AND PERSISTENCE .....                               | 27        |
| <b>5</b> | <b>CHAPTER 5: TOOL RESULTS AND EVALUATION .....</b>                 | <b>29</b> |
| 5.1      | EVALUATION SETUP .....                                              | 29        |
| 5.2      | MODEL COMPARISON .....                                              | 29        |
| 5.3      | CROSS-VALIDATION STABILITY .....                                    | 29        |
| 5.4      | HELD-OUT TEST PERFORMANCE .....                                     | 30        |
| 5.5      | PER-SOURCE EVALUATION .....                                         | 31        |
| 5.6      | FEATURE IMPORTANCE .....                                            | 31        |
| 5.7      | OPERATING POINT AND THRESHOLD ANALYSIS .....                        | 32        |
| 5.8      | EDUCATIONAL PLATFORM EVALUATION .....                               | 32        |
| 5.8.1    | <i>Professional Evaluation from Experts .....</i>                   | <i>33</i> |
| 5.9      | SUMMARY .....                                                       | 35        |
| <b>6</b> | <b>CHAPTER 6: ANALYSIS AND REFLECTION .....</b>                     | <b>36</b> |
| 6.1      | INTERPRETING SYSTEM OUTCOMES .....                                  | 36        |
| 6.2      | MISCLASSIFICATION ANALYSIS AND PER-SOURCE BIAS .....                | 36        |
| 6.3      | TECHNICAL CONSTRAINTS .....                                         | 36        |
| 6.4      | RELIABILITY OF FINDINGS AND CONFOUNDING FACTORS .....               | 37        |
| <b>7</b> | <b>CHAPTER 7: SUMMARY, CONCLUSIONS, AND FUTURE DIRECTIONS .....</b> | <b>38</b> |
| 7.1      | SUMMARY OF CONTRIBUTIONS .....                                      | 38        |
| 7.2      | FUTURE WORK .....                                                   | 38        |

|                         |           |
|-------------------------|-----------|
| <b>REFERENCES.....</b>  | <b>39</b> |
| <b>APPENDIX A. ....</b> | <b>41</b> |
| <b>APPENDIX B. ....</b> | <b>43</b> |
| <b>APPENDIX C. ....</b> | <b>44</b> |
| <b>APPENDIX D. ....</b> | <b>46</b> |
| <b>APPENDIX E. ....</b> | <b>50</b> |



# **1 Chapter 1: Introduction**

Email remains the dominant vector for cyberattacks against individuals and organisations. Phishing is the use of deceptive emails to manipulate recipients into disclosing credentials, transferring funds, or installing malware (Jabir et al., 2025). Phishing succeeds not because the underlying communication protocols are insecure but because it exploits the judgement of the person reading the message. Technical defences such as spam filters, domain authentication, and machine learning classifiers have become steadily more capable over the past decade, yet the rate of successful phishing attacks has continued to grow in parallel, reaching a cumulative total of over 4.7 million reported incidents in 2022 (Anti-Phishing Working Group, 2022). The persistence of the threat despite improvements in automated detection suggests that no purely technical defence can eliminate it: the human decision loop at the point of message receipt is an inseparable component of the attack surface.

A growing body of research argues that effective defence against phishing requires both a technical detection layer and a user-education layer, and that the two must reinforce each other rather than operate in isolation (Jabir et al., 2025; Alluqmani et al., 2025). A detection engine that produces only a binary verdict offers no opportunity for the user to learn from the analysis, and a training programme that runs independently of any live detection capability cannot connect its lessons to the messages a user actually encounters. Few published systems, however, implement both layers inside a single user-facing tool: the detection literature has focused on maximising classification accuracy on standard benchmarks, and the security awareness literature has focused on measuring the effectiveness of isolated training interventions.

## **1.1 Problem Statement**

The problem this project addresses is the lack of an integrated, explainable tool that combines automated phishing detection with interactive user education within a single, accessible interface. Three specific shortcomings in existing work motivate the design. First, the most widely cited public phishing email corpora conflate commercial spam with genuine social-engineering phishing, leading to classifiers that are formally evaluated against a mislabelled target (Alhuzali et al., 2025). Second, the detection literature reports almost exclusively aggregate metrics, which conceal systematic failures on specific phishing sub-genres and obscure the distributional biases introduced by training-corpus composition. Third, even the best-performing detection systems typically return only a risk score or binary verdict, offering a non-expert user no mechanism to understand which specific characteristics of a message triggered the warning or how to recognise similar messages in future.

## **1.2 Project Aims and Objectives**

The aim of this project is to design, implement, and evaluate a dual-function web application that provides both automated phishing detection and interactive user training, with all detection outputs grounded in human-readable explanations of the specific



indicators observed. The project is explicitly scoped as an academic final-year project rather than a commercial product; the emphasis is on methodological strictness, transparent evaluation, and a demonstrable prototype rather than on production hardening or market readiness. Within that scope, four objectives structure the work:

- i. Develop a detection engine capable of analysing user-submitted email content, including but not limited to headers, body text, and URLs. Then produces a phishing risk assessment supported by feature-level evidence.
- ii. Implement an educational platform that presents simulated phishing scenarios of increasing difficulty, delivers immediate, contextual feedback on user responses, and tracks per-session progress to reinforce learning.
- iii. Design a user interface that integrates both components inside a single coherent application, presents detection results alongside actionable recommendations, and maintains a consistent interaction model across the detection and training flows.
- iv. Evaluate the system meticulously: the detection engine against standard classification metrics on a held-out test partition with additional per-source analysis, and the educational platform against established usability heuristics.

### **1.3 Contributions**

Four specific contributions are claimed:

- i. First, the project audits all eleven publicly available email corpora commonly cited in the phishing detection literature and documents a principled rejection of those whose positive class conflates commercial spam with genuine phishing. The resulting training corpus is smaller than those reported in comparable studies, but it is methodologically more defensible for the specific prediction target of the work.
- ii. Second, the project reports per-source recall analysis on the held-out test partition, exposing a 5.78-percentage-point gap between advance-fee fraud and credential phishing that aggregate metrics would have concealed, and tracing the gap directly to the composition of the training corpus.
- iii. Third, the project implements a rule-based explanation engine that translates the engineered feature values underlying the classifier's decision into human-readable flags, recommendations, and a four-band verdict, grounded in the characterisation framework of Saka et al. (2024).
- iv. Fourth, the project integrates the detection engine with an interactive educational platform inside a single web application, realising the integrated-defence model called for by recent systematic reviews but rarely instantiated in working prototypes.

### **1.4 Scope and Limitations**

The project operates on user-submitted email content rather than on a live email stream: users paste or upload individual emails for analysis. This scope decision excludes several capabilities that would be expected in a commercial deployment:

- i. Real-time inbox monitoring or background scanning of incoming messages.
- ii. Any sort of integration with email clients (i.e. Gmail or Outlook).
- iii. Automatic blocking or quarantining of live messages on a mail server.

- iv. Anti-malware scanning of email attachments.

These exclusions are deliberate rather than incidental: each would substantially expand the engineering surface without advancing the core research question of whether integrated detection and education can be delivered effectively within a single accessible tool. The non-goals are documented in full in the original project brief, which is reproduced in Appendix A.

## **1.5 Project Significance**

This project offers value to both industry practitioners and the research community. From an industry perspective, the integrated detection-and-education design demonstrates that a lightweight, web-based tool can deliver both automated threat identification and formative user training without requiring enterprise infrastructure, email server integration, or user authentication. Thus, making it accessible to individuals, small organisations, and educational institutions that lack dedicated security operations teams. The explanation engine's human-readable flags provide a model for how detection outputs can be communicated to non-expert users in a way that builds understanding as opposed to dependence on the tool.

From a research perspective, the project contributes a documented methodology for auditing the publicly available phishing email corpora that support much of the detection literature, exposing a systematic conflation of commercial spam with genuine phishing that inflates reported detection accuracies. The per-source evaluation protocol introduced in this work provides a template for more transparent reporting of classifier performance across phishing sub-genres, addressing a recurring criticism of aggregate-only evaluation in the field. Together, these contributions are intended to be reusable independently of the specific classifier trained in this project.

## **1.6 Report Structure**

The remainder of this report is organised into six chapters. Chapter 2 reviews earlier studies on phishing detection, security awareness training, and phishing characterisation frameworks. Chapter 3 documents the methodology, covering the dataset audit, feature engineering, data partitioning, and model training. Chapter 4 describes the system design and implementation of both the detection engine and the educational platform. Chapter 5 presents the evaluation results for both components. Chapter 6 offers a critical discussion of the findings and their threats to validity, and Chapter 7 concludes with a summary of contributions and directions for future work.

## 2 Chapter 2: Literature Review

This chapter presents a critical review of prior work relevant to the development of the Phishing Detection and Awareness Tool. The review is organised around four themes. Section 2.1 establishes the scale and evolving character of the phishing threat, including the impact of generative artificial intelligence on attack sophistication. Section 2.2 surveys technical approaches to automated phishing detection proposed in the academic literature, spanning machine learning, deep learning and rule-based systems. Section 2.3 examines the human factors literature on security awareness training and its measured effectiveness in reducing user susceptibility. Section 2.4 reviews frameworks for characterising and explaining phishing emails, which inform the explanation engine component of the present system. Section 2.5 synthesises the limitations identified across these four threads and positions the contribution of this project within them.

### 2.1 The Phishing Threat Landscape

Phishing is a form of social engineering in which an attacker impersonates a trusted entity in electronic communication in order to elicit sensitive information. Examples include credentials, financial details, or authorisation from a targeted individual. Recent statistics underline the scale of the problem: the Anti-Phishing Working Group recorded more than 4.7 million phishing attacks in 2022, and the 2024 UK Cyber Security Breaches Survey, cited by Jabir et al. (2025), reports that phishing remains the single most common attack vector encountered by organisations.

Three sub-genres of phishing are relevant to the present work. Credential phishing, the dominant modern subtype, uses brand-impersonation templates (PayPal, Apple, banking portals) to direct victims to cloned login pages. Advance-fee fraud, colloquially known as the Nigerian scam, uses elaborate narratives about inheritance, lottery winnings, or stranded travellers to extract money rather than credentials; it is stylistically distinctive and was historically the most abundant form in academic corpora. Spear phishing targets specific individuals using personal information harvested from public sources and, because it is inherently lower-volume, is under-represented in the public corpora used for training detection systems.

The emergence of generative artificial intelligence has reshaped this landscape in ways that directly constrain the claims that can be made about any detector trained on historical data. Jabir et al. (2025), in a systematic review of 116 papers, argue that large language models have substantially lowered both the cost and the linguistic sophistication barrier to mounting convincing phishing campaigns. Qi et al. (2025) demonstrate a framework called SpearBot in which an LLM is used in a generate-and-critique loop to produce spear-phishing emails that empirically evade manually trained detectors. The implication for the present work is that any classifier trained on pre-2020 corpora can only be evaluated as a baseline against the established threat, not as a tool with proven effectiveness against AI-augmented contemporary attacks. This constraint is acknowledged explicitly in the critical discussion of Chapter 6.

### 2.2 Technical Approaches to Phishing Detection

Research on automated phishing detection has seen evolution through numerous overlapping generations. Rule-based systems were the core for earlier systems, relying on

curated heuristics applied to email headers and URL structure (for example, mismatched Reply-To domains, IP-literal hosts, or blacklisted domain patterns). Such systems are transparent and fast but brittle in the face of novel attack patterns and are progressively less effective as attackers learn to avoid individual triggers. Modern surveys, such as Alhuzali et al. (2025), treat rule-based systems primarily as a baseline rather than as a viable production technique.

The second generation applies classical machine learning to hand-engineered feature vectors extracted from emails or URLs. Adeyemi Onih (2024) compared K-Nearest Neighbours, Artificial Neural Networks, and Random Forest on URL and website features, reporting 94–96% accuracy among the three, and demonstrated practical integration of the best-performing model into a Django web application. Naz et al. (2024), in a comprehensive survey of social engineering attacks, examine feature sets for XGBoost, Gradient-Boosted Decision Trees, and Support Vector Machines, and argue that tree-ensemble methods offer the most favourable combination of accuracy and interpretability for production deployment. Random Forest, in particular, is consistently reported as a strong baseline across both the URL and email classification literature, and its Gini-based feature importance scores (a measure of how much each feature contributes to reducing classification error across the ensemble's decision trees) provide a native mechanism for explaining individual predictions. This property, as described in Chapter 4 of the present work, is directly exploited by the explanation engine.

The third generation applies deep learning directly to raw input representations. Aldakheel et al. (2023) reported 98.77% accuracy using a Convolutional Neural Network (CNN) on URL features, and Aljofey et al. (2020) achieved 97.4% with a character-level CNN operating on raw URL sequences. The most comprehensive recent benchmark evaluated fourteen Machine Learning (ML) and Deep Learning (DL) models across ten public email corpora; their transformer-based architectures (BERT and RoBERTa) achieved 98.99% and 99.08% accuracy, respectively, outperforming classical ML baselines by 4.7 percentage points, Alhuzali et al. (2025).

**Table 2.1:** *Representative phishing detection studies compared on approach, dataset, and reported performance.*

| Study                   | Approach            | Dataset                                   | Reported accuracy / F1       |
|-------------------------|---------------------|-------------------------------------------|------------------------------|
| Aldakheel et al. (2023) | URL-only CNN        | 10,000 PhishTank URLs + 10,000 legitimate | 98.77% accuracy              |
| Aljofey et al. (2020)   | Character-level CNN | URL character sequences                   | 97.4% accuracy               |
| Adeyemi Onih (2024)     | KNN, ANN, RF        | URL and website features                  | 94–96% accuracy              |
| Alhuzali et al. (2025)  | 14 ML/DL models     | Ten public email corpora, merged          | RoBERTa 99.08% on merged set |
| Naz et al. (2024)       | XGBoost, GBDT, SVM  | Social-engineering indicator features     | 89–95% accuracy              |

| Study               | Approach                 | Dataset                                       | Reported accuracy / F1                             |
|---------------------|--------------------------|-----------------------------------------------|----------------------------------------------------|
| Pohan et al. (2023) | Simulated phishing       | Student academic emails                       | 70% initial susceptibility, improved post-training |
| Present work        | RF + engineered + TF-IDF | Audited five-corpus email set (27,180 emails) | 97.18% F1, per-source gap reported                 |

Three observations motivate this project's methodology. First, the literature overwhelmingly focuses on URL classification; the present work focuses on email content because the dual-purpose design requires processing header, body, and URL signals simultaneously. Second, reported accuracies above 98% on balanced datasets should be interpreted with caution, as aggregate metrics conceal sub-genre failures (Chapter 5). Third, Alhuzali et al. (2025) document but then absorb the fact that several widely used corpora label commercial spam as phishing; the present work departs from this practice and treats the distinction as methodologically load-bearing (Chapter 3).

## 2.3 Security Awareness Training

Technical detection alone is insufficient because phishing fundamentally exploits human judgement. Jabir et al. (2025) and Alluqmani et al. (2025) both argue that effective defence requires both a technical detection layer and a human awareness layer designed to reinforce each other. That is the foundational premise of the present tool's dual-function design. Tschakert and Ngamsuriyaroj (2019) compared classroom instruction with video, game, and text-based training in a 2×2 mixed-factorial design and found that all formats significantly reduced phishing susceptibility, but no single combination was statistically superior. This finding directly informs the decision to offer progressive difficulty tiers rather than a single training module. Ahmad et al. (2023) developed a web-based application combining simulation-based training with embedded learning tools delivered in-context, demonstrating that the combination improved detection ability more reliably than simulation alone and emphasising the importance of immediate explanatory feedback.

The second design principle is shaping the present educational platform. Alluqmani et al. (2025) synthesised over thirty studies (2021–2024) and identified three consistent conclusions: simulation-based training significantly reduces susceptibility, its effectiveness depends on adaptivity and reinforcement over time, and demographic and personality factors materially influence both vulnerability and training response. Pohan et al. (2023) reported that 70.6% of students clicked on simulated phishing emails prior to training, and that susceptibility reduced substantially after a brief awareness intervention. Rizzoni et al. (2022) found that personalised phishing emails were markedly more successful than generic ones in a large hospital simulation, underlining the importance of scenario realism.

**Table 2.2:** *Representative security awareness training studies and their principal findings.*

| Study                              | Training format                      | Principal finding                                                                                 |
|------------------------------------|--------------------------------------|---------------------------------------------------------------------------------------------------|
| Tschakert and Ngamsuriyaroj (2019) | Video, game, text, classroom         | All formats reduce false-negative rate; no single combination statistically superior.             |
| Ahmad et al. (2023)                | Simulation + embedded learning tools | Integrating simulation with in-context learning improved detection and engagement.                |
| Alluqmani et al. (2025)            | Review of 30+ SAT studies            | Simulation-based SAT is most effective when adaptive, behaviour-aware, and reinforced over time.  |
| Pohan et al. (2023)                | University email simulation          | 70.6% of students clicked on phishing emails prior to training; awareness improved post-exposure. |
| Rizzoni et al. (2022)              | Hospital-wide simulation             | Personalised phishing emails far more effective than generic ones; scenario realism matters.      |

The present work adopts the consensus established by these studies: scenario-based simulations across three difficulty tiers with immediate contextual feedback and session-based progress tracking. The educational platform is assessed heuristically rather than empirically (Chapter 5), and a formal pre-/post-intervention study is identified as future work.

## 2.4 Phishing Characterisation and Explanation

A detection system that outputs only a binary label provides no affordance for user learning. Saka et al. (2024) address this shortcoming directly by proposing the Phishing Codebook, an organized framework for the qualitative characterisation of phishing emails along axes including action requested (click, download, reply, call), urgency cues, threat cues, and main topic. The codebook was developed through manual analysis of 503 phishing emails and is intended to normalise the highly variable structure of real phishing messages into a consistent representation suitable for both human review and machine analysis. The authors specifically observe that purely statistical text-classification approaches struggle when phishing emails share vocabulary across campaigns or when individual emails are extremely short - both conditions under which the Term frequency-inverse document frequency (TF-IDF) component of the present system is expected to produce low-confidence outputs.

The relevance of Saka et al. (2024) to the present work is twofold. First, it provides academic justification for the explanation engine's rule-based design described in Chapter 4: the engine generates flags that map onto the codebook's characterisation axes (urgency, credential request, financial lure, suspicious URL structure) rather than attempting to surface the internal decision path of the Random Forest classifier. Second, it offers a structural observation that is borne out by the feature-importance analysis reported in

Chapter 5: classifiers trained on heterogeneous corpora will reflect the vocabulary of the dominant sub-genre, and explanation systems must therefore be designed to cover the full indicator space independently of which specific features happened to be weighted highly by the trained model. This separation of classification-time weights from explanation-time coverage is a direct methodological consequence of the Saka et al. framework.

## **2.5 Findings from the Literature Review**

Four unaddressed areas emerge from the preceding review and together motivate the contributions of the present work. First, detection research and awareness-training research remain fragmented: detection engines flag threats without teaching, and awareness platforms train users without offering live analysis. Jabir et al. (2025) and Alluqmani et al. (2025) both call for integrated defence, but few published prototypes implement this in a single tool. Second, several widely cited public email corpora conflate commercial spam with genuine phishing in their positive class; Alhuzali et al. (2025) note the distinction but merge the conflated corpora anyway. Third, the dominant evaluation protocol reports only aggregate metrics, masking systematic failures on specific phishing sub-genres. Fourth, detection systems typically return a binary verdict or risk score without explaining which observable characteristics triggered the warning; Saka et al. (2024) offer a characterisation framework suitable for human-readable output, but it has not previously been combined with a working classifier inside a user-facing tool. The present project addresses all four areas: it integrates detection and education in a single application, audits and rejects the conflated corpora, reports per-source recall alongside aggregate metrics, and implements an explanation engine grounded in the Saka et al. framework.

### 3 Chapter 3: Proposed Methodology

This chapter documents the methodology used to build the detection component of the Phishing Detection and Awareness Tool, covering the dataset audit, feature engineering, data partitioning and class-imbalance handling, model selection and hyperparameter tuning, the end-to-end training pipeline, and artefact export.

#### 3.1 Methodology Framework

The project followed an iterative, phase-based development methodology, with work organised into incremental phases: dataset preparation, feature engineering, model training, backend integration, frontend development, and educational platform. Each phase was reviewed during weekly supervisor meetings, which provided regular checkpoints for assessing progress and adjusting priorities. The ML training pipeline was developed in Google Colab to enable rapid experimentation, with the final trained artefacts exported and integrated into the web application only after evaluation on the held-out test partition confirmed acceptable performance. The Gantt chart in Appendix B compares the planned and actual timelines.

#### 3.2 Datasets

The single most consequential methodological decision in this project was which public email corpora to include in the training data. Eleven corpora commonly cited in the phishing detection literature were obtained and audited by manually sampling the positive class of each: Nazario\_5, Nigerian\_Fraud, Nigerian\_5, Enron, SpamAssassin, Ling, CEAS\_08, TREC-05, TREC-06, TREC-07, and the Kaggle Phishing\_Email corpus. Of these, only two were found to contain genuine phishing emails in their positive class. The remaining nine either labelled commercial spam as phishing, duplicated legitimate rows already present in other corpora, or failed to parse as valid CSV input. Table 3.1 summarises the audit.

**Table 3.1:** Audit of the eleven publicly available email corpora considered for the training set.

| Corpus         | Verdict  | Reason                                                                                                      |
|----------------|----------|-------------------------------------------------------------------------------------------------------------|
| Nazario_5      | Accepted | Genuine credential phishing targeting PayPal, Apple, eBay, and banking brands.                              |
| Nigerian_Fraud | Accepted | Genuine advance-fee social engineering; stylistically distinct from credential phishing but still in scope. |
| Enron          | Partial  | label=0 rows retained as legitimate email; label=1 rows contained product spam and were discarded.          |
| SpamAssassin   | Partial  | label=0 rows retained as legitimate email; label=1 rows contained commercial spam and were discarded.       |



| Corpus                | Verdict  | Reason                                                                                                            |
|-----------------------|----------|-------------------------------------------------------------------------------------------------------------------|
| Ling                  | Partial  | label=0 rows retained as legitimate email; label=1 rows contained early-2000s commercial spam and were discarded. |
| Nigerian_5            | Rejected | Legitimate partition duplicates Nazario_5 legitimate rows; using it would double-count the same 1,175 emails.     |
| CEAS_08               | Rejected | label=1 rows are commercial spam (celebrity gossip, replica watches), not phishing.                               |
| TREC_05               | Rejected | Malformed CSV with label-column text leakage; positive class is spam, not phishing.                               |
| TREC_06               | Rejected | Positive class is commercial spam (loan advertisements, generic marketing).                                       |
| TREC_07               | Rejected | Malformed CSV parsing: positive class is spam, not phishing.                                                      |
| Kaggle Phishing_Email | Rejected | Positive class is adult spam and pump-and-dump stock spam; no headers or sender fields.                           |

This finding is not unique to the present project. Alhuzali et al. (2025) surveyed the same nine non-phishing corpora in a recent systematic study of phishing email detection and acknowledged the spam/phishing distinction in their dataset description but subsequently merged all positive-class rows into a single 'phishing' target for training purposes. The present work departs from that practice and treats the distinction as methodologically load-bearing: because the project's stated purpose is the detection of phishing and social-engineering attacks (Chapter 1), training the classifier on commercial spam would systematically mislabel the prediction target and contaminate every downstream metric. The decision to reject the conflated corpora is therefore presented not as a limitation but as a deliberate methodological choice, with the associated limitation, a smaller positive class dominated by the more abundant Nigerian\_Fraud subset, analysed in Chapter 6.

The final corpus contained 27,180 deduplicated emails after a subject+body deduplication pass removed exact repeats. The phishing class consisted of 1,565 credential-phishing emails from Nazario\_5 and 3,332 advance-fee fraud emails from Nigerian\_Fraud (4,897 positives, 18.0% of the corpus). The legitimate class consisted of 15,791 emails from Enron, 4,091 from SpamAssassin, and 2,401 from Ling (22,283 negatives, 82.0% of the corpus). All rows were reduced to a common schema of seven columns - sender, receiver, date, subject, body, URLs, label and a source-origin column was retained to support the per-source evaluation reported in Section 5.5.

### 3.3 Feature Engineering

Each email is represented as a fixed-length numeric feature vector combining two complementary feature groups: 22 hand-engineered features drawn from the email header, body, and URL structure, and 300 TF-IDF features derived from the subject line (100 terms) and body text (200 terms). The engineered features encode domain knowledge about phishing indicators that the literature has consistently found discriminative, while

the TF-IDF features provide a data-driven representation of the vocabulary most associated with each class. The resulting 322-dimensional feature vector is passed to a single classifier rather than an ensemble of modality-specific sub-models, because treating all features as a flat vector enabled Random Forest to automatically discover interactions between vocabulary and structural signals.

Table 3.2 enumerates the 22 engineered features. The header group (5 features) captures sender authenticity signals: a mismatch between the From-header domain and a Reply-To domain recovered from the raw body text, SPF and DKIM authentication verdicts when present, the number of Received relay hops, and a display-name spoofing flag that triggers when the From header names a known brand (PayPal, Amazon, Apple, Microsoft, Google, Facebook, Netflix) whose canonical domain does not match the sending domain. The body group (9 features) captures content-level indicators, including an urgency-keyword count over a curated vocabulary of 20 phishing-associated terms, HTML-to-text ratio, URL density, average word length, exclamation-mark count, and binary flags for credential-request and advance-fee language. The URL group (8 features) characterises the first URL observed in each email - length, dot count, subdomain depth, HTTPS usage, IP-literal host, embedded @ symbol, suspicious TLD membership - plus a final feature counting the number of unique registered domains referenced across all URLs in the email.

**Table 3.2:** *The 22 engineered features extracted from each email, grouped by source.*

| Feature No. | Feature               | Group  | Description                                                                                        |
|-------------|-----------------------|--------|----------------------------------------------------------------------------------------------------|
| 1           | sender_reply_mismatch | Header | Binary flag: the From-header domain differs from a Reply-To domain extracted from the body text.   |
| 2           | has_spf               | Header | Binary flag: an SPF=pass verdict appears in the raw email.                                         |
| 3           | has_dkim              | Header | Binary flag: a DKIM=pass verdict appears in the raw email.                                         |
| 4           | received_hops         | Header | Count of Received: lines observed, indicating the number of relay hops.                            |
| 5           | display_name_spoofed  | Header | Binary flag: the display name names a known brand whose official domain is not the sender domain.  |
| 6           | urgency_keyword_count | Body   | Total occurrences of any of twenty urgency keywords (e.g. verify, suspended, act now, click here). |
| 7           | has_html              | Body   | Binary flag indicating the presence of HTML tags or anchor elements.                               |
| 8           | html_to_text_ratio    | Body   | Fraction of body characters that are HTML markup rather than readable text.                        |

| Feature No. | Feature               | Group | Description                                                                                                |
|-------------|-----------------------|-------|------------------------------------------------------------------------------------------------------------|
| 9           | url_count             | Body  | Total number of URLs detected in the body text.                                                            |
| 10          | body_url_ratio        | Body  | Fraction of body characters consumed by URL strings.                                                       |
| 11          | avg_word_length       | Body  | Mean length of alphabetic words in the body.                                                               |
| 12          | exclamation_count     | Body  | Number of exclamation marks in the body text.                                                              |
| 13          | credential_request    | Body  | Binary flag: body or subject contains password, login, credentials, SSN, or credit-card language.          |
| 14          | financial_threat      | Body  | Binary flag: body or subject contains advance-fee vocabulary (wire transfer, inheritance, lottery, prize). |
| 15          | url_uses_ip           | URL   | Binary flag: first URL uses a raw IP-literal host rather than a domain name.                               |
| 16          | url_has_at_symbol     | URL   | Binary flag: first URL contains an @ character, a known redirection technique.                             |
| 17          | url_length            | URL   | Character length of the first URL in the email.                                                            |
| 18          | url_dot_count         | URL   | Number of dots in the first URL.                                                                           |
| 19          | url_has_https         | URL   | Binary flag: first URL uses the HTTPS scheme.                                                              |
| 20          | url_subdomain_depth   | URL   | Number of subdomain segments in the first URL.                                                             |
| 21          | url_suspicious_tld    | URL   | Binary flag: first URL uses a TLD from a curated suspicious set (tk, ml, ga, xyz, top, club, work, etc.).  |
| 22          | external_domain_count | URL   | Number of unique registered domains linked across all URLs in the email.                                   |

The TF-IDF features are computed using two independently fitted `TfidfVectorizer` instances, one for subject lines (`max_features = 100`) and one for body text (`max_features = 200`), both configured with `n-gram range (1, 2)`, minimum document frequency 0.005, maximum document frequency 0.95, and a combined stopwords list containing scikit-learn's English stopwords set augmented with 17 domain-specific tokens (enron, ect, kaminski, spamassassin, linguist, mbox, and similar). The domain stopwords were added to prevent the vectorisers from learning dataset-identifying fingerprints rather than

phishing-indicative language. Before vectorisation, body text is cleaned by stripping HTML tags, replacing URLs with a sentinel URL token, and collapsing whitespace. The cleaning function is defined once in the training notebook and imported verbatim by the inference pipeline to guarantee that inference-time vectors are byte-identical to training-time vectors.

A critical methodological point concerns where the vectorisers are fitted. Both `TfidfVectorizer` instances are fitted exclusively on the training partition after splitting, not on the full corpus. Fitting them on the full corpus before splitting would allow test-set vocabulary to influence the vectoriser's vocabulary selection and inverse-document-frequency weights, leaking information from the held-out partition into the trained model. By fitting on the training data only and then transforming the validation and test partitions with the same fitted vectoriser, the evaluation reported in Chapter 5 measures the model's performance on genuinely unseen vocabulary rather than on text that has already contributed to the feature representation. A post-fitting sanity check confirms that no dataset-identifying tokens (`enron`, `kaminski`, `spamassassin`) appear in the learned vocabularies.

### 3.4 Data Partitioning and Class Imbalance

The combined corpus was split into training, validation, and test partitions using a two-stage stratified split (a partitioning method that preserves the original class ratio in every resulting subset). In the first stage, 20% of the corpus was withheld as the final test partition (5,436 emails) and locked away; this partition was not touched again until the final evaluation reported in Section 5.5. In the second stage, the remaining 80% was split into a 75/25 training/validation partition (16,308 and 5,436 emails, respectively), giving an overall 60/20/20 split. Both stages used stratified sampling on the label column, with a fixed random seed of 42, to preserve the phishing class ratio across partitions.

The positive class accounts for 18.0% of the corpus, a modest but non-negligible imbalance. To counteract this imbalance during training, the Synthetic Minority Oversampling Technique (SMOTE) of Chawla et al. (2002) was applied to the training partition only, synthesising new minority-class examples in feature space until the training set was balanced. Two design choices are important here. First, SMOTE is applied after the train/val/test split rather than before, so that synthetic examples generated from training minority points cannot leak into the validation or test partitions. Second, SMOTE is applied after feature extraction rather than on raw text, because oversampling in raw-text space is ill-defined. Applying SMOTE in the 322-dimensional feature space produces interpolated feature vectors that are consistent with the feature space the classifier ultimately operates on.

**Table 3.3:** *The class distribution of the training partition before and after SMOTE application.*

| Stage        | Legitimate (label = 0) | Phishing (label = 1) | Total  |
|--------------|------------------------|----------------------|--------|
| Before SMOTE | 13,341                 | 2,967                | 16,308 |
| After SMOTE  | 13,341                 | 13,341               | 26,682 |

The validation and test partitions were not oversampled and retain the original 82/18 class ratio, ensuring that all reported metrics reflect the classifier's performance on naturally distributed data rather than on synthetically balanced data.

### 3.5 Model Selection and Hyperparameter Tuning

Three classifiers were compared on the validation partition using the SMOTE-balanced training set: Logistic Regression (max\_iter = 1000, default L2 regularisation), Random Forest (n\_estimators = 100), and XGBoost (n\_estimators = 100, binary logistic objective). All three were evaluated on accuracy, F1-score, and ROC-AUC (the area under the receiver operating characteristic curve, measuring class separability) against the validation partition; the test partition remained untouched. The results are reported in Table 5.1 of Chapter 5. Random Forest was selected as the primary classifier; the full justification, including per-model metrics and architectural reasoning, is presented in Section 5.2.

With the classifier family chosen, hyperparameters were tuned using RandomizedSearchCV on the combined training and validation partitions (21,744 emails), with SMOTE applied to the combined partition before tuning to preserve consistency with the training protocol. The search used three-fold cross-validation, optimised F1-score, and drew ten random configurations from the search space shown in Table 3.4. Ten iterations were chosen as a compromise between search coverage and training cost: each fit of the 322-feature Random Forest over a SMOTE-balanced training set required several minutes in the Google Colab environment, so a larger search would have exceeded the available session limits.

*Table 3.4: RandomizedSearchCV search space for Random Forest hyperparameter tuning.*

| Hyperparameter    | Search values    |
|-------------------|------------------|
| n_estimators      | 100, 200, 300    |
| max_depth         | None, 10, 20, 30 |
| min_samples_split | 2, 5, 10         |
| min_samples_leaf  | 1, 2, 4          |
| max_features      | sqrt, log2       |

The tuned configuration selected by the search was n\_estimators = 100, max\_depth = None, min\_samples\_split = 10, min\_samples\_leaf = 2, and max\_features = 'sqrt', with a best cross-validated F1 of 0.9624. After tuning, the final estimator was evaluated exactly once on the held-out test partition to produce the metrics reported in Section 5.5. This single-evaluation discipline is important: repeated evaluation of tuned models against the test set, even informally, constitutes a soft form of test-set contamination in which the experimenter's choices begin to reflect test-set idiosyncrasies. Restricting test-set access to a single evaluation preserves the statistical validity of the reported metrics.

### 3.6 Training Pipeline

Algorithm 1 presents the end-to-end training pipeline as executed in the phishing\_detection\_pipeline\_v2 Jupyter notebook. The pipeline combines the dataset

construction, feature engineering, partitioning, oversampling, model training, and export steps into a single linear flow.

**Algorithm 1: End-to-end training pipeline.**

```

Input:  five labelled CSV corpora in DATASET_DIR
Output: four serialised artefacts (phishing_model.pkl,
tfidf_subject.pkl,
      tfidf_body.pkl, feature_names.pkl)

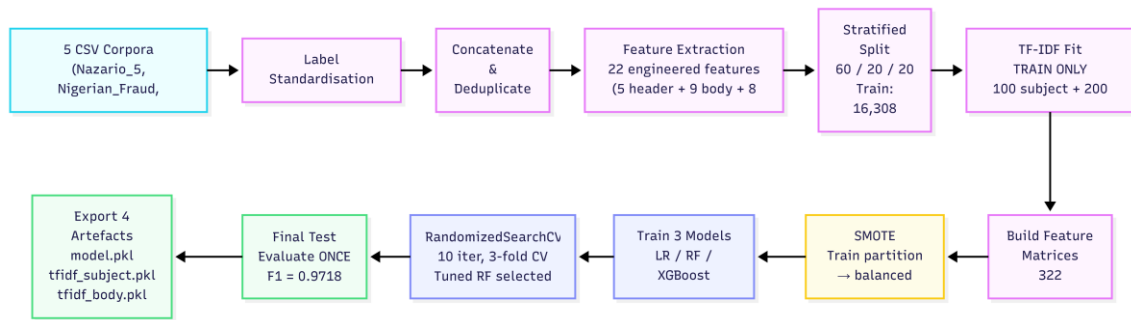
1.  for each (filename, keep_label) in phishing U legitimate
sources:
      df <- read_csv(filename)
      df <- standardise_labels(df)
      df <- df[df.label == keep_label]
      frames.append(df)
2.  df_raw <- concat(frames)
3.  df_raw <- drop_duplicates(df_raw, subset=[subject, body])
4.  for each row in df_raw:
      row.features <- extract_header_features(row)
                      U extract_body_features(row)
                      U extract_url_features(row)
5.  df_engineered <- build feature DataFrame from row.features
6.  (X_trainval, X_test, y_trainval, y_test)
    <- stratified_split(df_engineered, test_size=0.20, seed=42)
    (X_train, X_val, y_train, y_val)
    <- stratified_split(X_trainval, test_size=0.25, seed=42)
7.  tfidf_subject <- TfidfVectorizer(max=100,
ngram=(1,2)).fit(X_train.subject)
    tfidf_body    <- TfidfVectorizer(max=200,
ngram=(1,2)).fit(X_train.body)
8.  X_train_mat <- build_feature_matrix(X_train)
    X_val_mat   <- build_feature_matrix(X_val)
    X_test_mat  <- build_feature_matrix(X_test)
9.  (X_train_bal, y_train_bal) <-
SMOTE(seed=42).fit_resample(X_train_mat, y_train)
10. for each model in {LogReg, RandomForest, XGBoost}:
      model.fit(X_train_bal, y_train_bal)
      evaluate(model, X_val_mat, y_val)
11. search <- RandomizedSearchCV(RandomForest, param_dist,
n_iter=10,
                                cv=3, scoring='f1')
      search.fit(SMOTE(X_trainval_mat, y_trainval))
12. best_estimator <- search.best_estimator_
13. evaluate(best_estimator, X_test_mat, y_test) // run ONCE only
14. joblib.dump(best_estimator, 'phishing_model.pkl')
    joblib.dump(tfidf_subject,  'tfidf_subject.pkl')
    joblib.dump(tfidf_body,     'tfidf_body.pkl')
    joblib.dump(FEATURE_COLUMNS, 'feature_names.pkl')

```

Each step is implemented as an independently runnable notebook cell, allowing intermediate results to be inspected during development. The final cell exports the four serialised artefacts consumed by the Flask backend at runtime.

**Table 3.5:** Compiled dataset statistics after preparation, splitting, and feature extraction.

| Statistic                          | Value                   |
|------------------------------------|-------------------------|
| Total emails (after deduplication) | 27,180                  |
| Phishing emails (label = 1)        | 4,897 (18.0%)           |
| Legitimate emails (label = 0)      | 22,283 (82.0%)          |
| Training partition                 | 16,308 (60%)            |
| Validation partition               | 5,436 (20%)             |
| Test partition                     | 5,436 (20%)             |
| Training partition after SMOTE     | 26,682 (balanced 50/50) |
| Engineered features                | 22                      |
| Subject TF-IDF features            | 100                     |
| Body TF-IDF features               | 200                     |
| Total feature vector width         | 322                     |
| Source corpora accepted            | 5 of 11 audited         |



**Figure 3.1:** End-to-end training pipeline from raw CSV corpora to serialised model artefacts.

### 3.7 Artefact Export

The trained pipeline is persisted as four artefacts: the tuned Random Forest classifier (phishing\_model.pkl), two fitted TF-IDF vectorisers (tfidf\_subject.pkl and tfidf\_body.pkl), and the ordered list of 322 feature column names (feature\_names.pkl). Three mechanisms guard against training–serving drift: the feature extraction functions used at training time are imported verbatim by the inference pipeline, the TF-IDF vectorisers are serialised with their fitted state intact, and the inference pipeline reindexes its feature dictionary against feature\_names.pkl before passing the vector to the classifier.

## 4 Chapter 4: Architecture and Technical Implementation

This chapter describes the design and implementation of the Phishing Detection and Awareness Tool as a running web application, covering the three-tier architecture, backend services, inference pipeline, explanation engine, educational platform, frontend, and database schema.

### 4.1 Architectural Overview

The system is implemented as a three-tier web application comprising a React single-page frontend, a Python Flask REST backend, and a SQLite persistence layer. The tiers communicate over HTTP using JSON payloads, with the frontend issuing requests against a stable REST interface rooted at `/api`. This separation ensures that the user interface can be developed and tested independently of the detection pipeline, and that the trained machine learning model is accessed only through a single well-defined service boundary rather than being coupled to any specific user interface technology.

A single detection request follows a well-defined path through the application. The user submits raw email content from the React detection page, either by pasting text into a form or by uploading a `.eml` file. The frontend forwards the content to the Flask backend via a POST request to `/api/analyse`. The detection blueprint handler validates the input, hands the raw string to the email parser to produce a structured dictionary, delegates feature extraction and classification to the ML classifier service, passes the resulting engineered feature dictionary and model probability to the explanation engine, and finally serialises the response as JSON. The frontend then renders the risk score, verdict banner, threat flags, and recommendations inside the analysis results panel.

### 4.2 Backend Services

The backend follows the Flask application factory pattern, which enables isolated unit testing and avoids circular import issues. The factory performs one-time initialisation at startup, including loading the trained ML artefacts and registering three API blueprints (health, detection, and education) under the `/api` URL prefix. The detection pipeline is decomposed into four service modules with strictly bounded responsibilities, summarised in Table 4.1.

**Table 4.1:** Responsibilities of the backend service modules that implement the detection pipeline.

| Service module                    | Responsibility                                                                                                                                      |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>email_parser.py</code>      | Parses raw <code>.eml</code> content, pasted text with headers, or plain-text fallback into a flat dictionary of sender, subject, body, and URLs.   |
| <code>feature_extractor.py</code> | Computes the 22 engineered features (5 header, 9 body, 8 URL) used during training, plus the body text cleaner for TF-IDF.                          |
| <code>ml_classifier.py</code>     | Loads the four serialised artefacts once at startup and exposes <code>classify()</code> , which runs the full inference pipeline on a parsed email. |

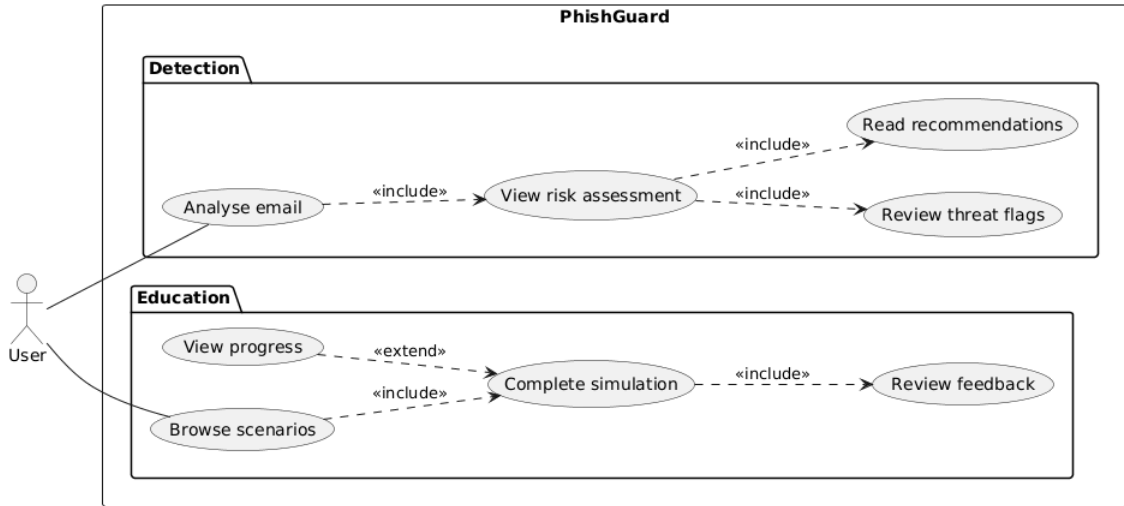


| Service module | Responsibility                                                                                                                                       |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| explanation.py | Translates the engineered feature dictionary and the model probability into human-readable flags, a risk score, a verdict band, and recommendations. |

This decomposition ensures that the feature extractor is imported from exactly one place, guaranteeing that inference-time behaviour matches training time. The ML classifier uses a singleton pattern (ensuring only one instance of the loaded model exists in memory) to load artefacts once at startup, with a consistency check verifying that the model's expected input width matches the feature-names list. If any artefact is missing, the `/api/analyse` endpoint returns an HTTP 503 status code while the rest of the application continues operating. The REST interface is summarised in Table 4.2 below:

**Table 4.2:** REST API endpoints exposed by the Flask backend.

| Method | Route                                         | Purpose                                                                                                                             |
|--------|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| GET    | <code>/api/health</code>                      | Liveness check: reports whether the ML model is loaded and how many scenarios are seeded.                                           |
| POST   | <code>/api/analyse</code>                     | Accepts raw email text or a .eml file upload; returns the full analysis response described in Section 4.5.                          |
| GET    | <code>/api/scenarios</code>                   | Lists scenarios, optionally filtered by difficulty, with answer fields withheld.                                                    |
| GET    | <code>/api/scenarios/&lt;id&gt;</code>        | Retrieves one scenario by identifier, again with ground-truth answer fields withheld.                                               |
| POST   | <code>/api/scenarios/&lt;id&gt;/answer</code> | Submits a user answer; records the attempt against the anonymous session and returns correctness, explanation, and learning points. |
| GET    | <code>/api/progress/&lt;session_id&gt;</code> | Aggregates per-session progress statistics for the learning dashboard.                                                              |



**Figure 4.1:** Use case diagram showing the two primary interaction workflows available to the user.

### 4.3 Inference Pipeline

Algorithm 2 presents the full inference pipeline executed on each POST to `/api/analyse`. The algorithm combines the four service modules into a single sequential flow: email parsing, engineered feature extraction, TF-IDF vectorisation, feature-vector reindexing, model inference, and explanation generation. Each step is idempotent and stateless with respect to the request, so concurrent requests share no mutable state beyond the loaded model singletons.

**Algorithm 2: Email analysis pipeline.**

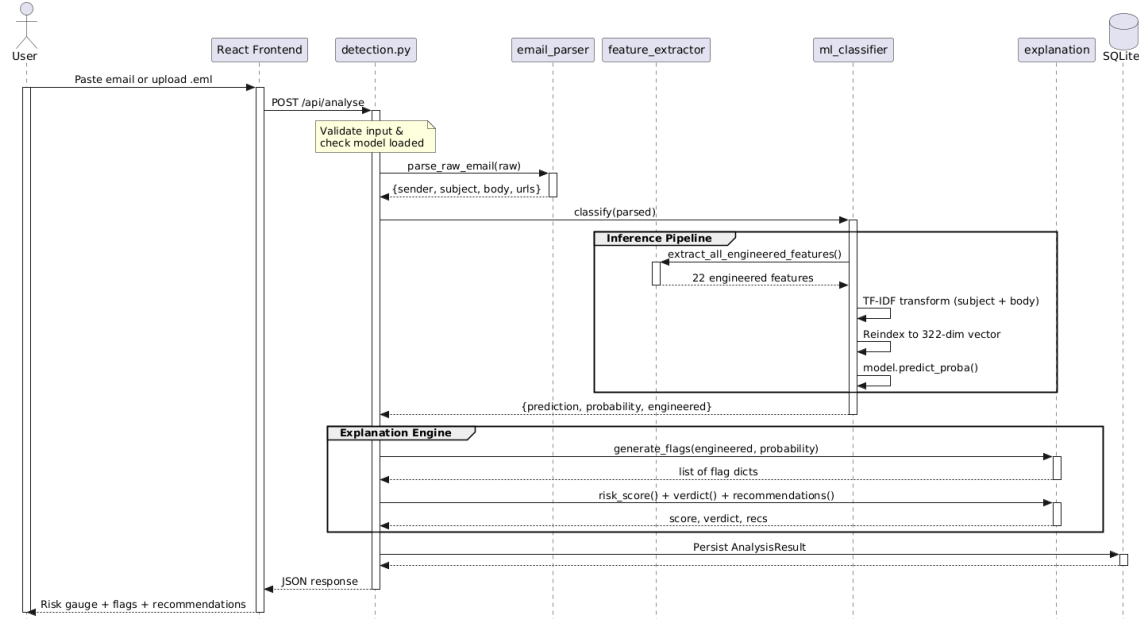
```

Input:  raw_email  - raw email content (string from paste or .eml
                  upload)
Output: response   - JSON object with risk score, verdict, flags,
                  and recommendations

1.  parsed <- parse_raw_email(raw_email)
    // → {sender, subject, body, urls}
2.  engineered <- extract_all_engineered_features(parsed)
    // → 22 header/body/URL features
3.  features <- copy of engineered
4.  subj_vec <- tfidf_subject.transform([parsed.subject])
5.  body_vec <-
    tfidf_body.transform([clean_body_for_tfidf(parsed.body)])
6.  for each (name, value) in subj_vec:  features["subj_" + name] <-
    value
7.  for each (name, value) in body_vec:  features["body_" + name] <-
    value
8.  x <- pandas.Series(features).reindex(feature_columns).fillna(0)
    // → 322-wide vector in the training column order
9.  prediction <- model.predict(x)
10. probability <- model.predict_proba(x)[1]
11. flags <- generate_flags(engineered, probability)
12. score <- round(probability * 100)
13. verdict <- verdict_band(probability)
14. recommendations <- summary_recommendations(flags, probability)
15. return {score, verdict, probability, flags, recommendations,
    ...}

```

The reindex step at line 8 ensures the feature vector is always in the training column order by reindexing against `feature_names.pkl`, filling any missing entries with zero. This guarantees both correct ordering and graceful degradation if a single extractor fails.



**Figure 4.2:** Sequence of service calls for a single `POST /api/analyse` request, showing the complete parse → extract → classify → explain pipeline.

## 4.4 Explanation Engine

The machine learning classifier produces a phishing probability, but a bare probability is not useful to a non-expert user. The explanation engine bridges this gap by translating the engineered feature dictionary into human-readable flags that explain, in ordinary language, which specific characteristics of the email triggered the model's concern. The engine is deliberately rule-based rather than model-driven. It inspects the engineered feature values directly and generates one flag per observed suspicious pattern. Examples include: a mismatched reply-to domain, an IP-literal URL, the presence of credential-request keywords, or an unusually high HTML-to-text ratio, among others. Each flag carries a severity level, a short description of the finding, and an actionable recommendation for the user.

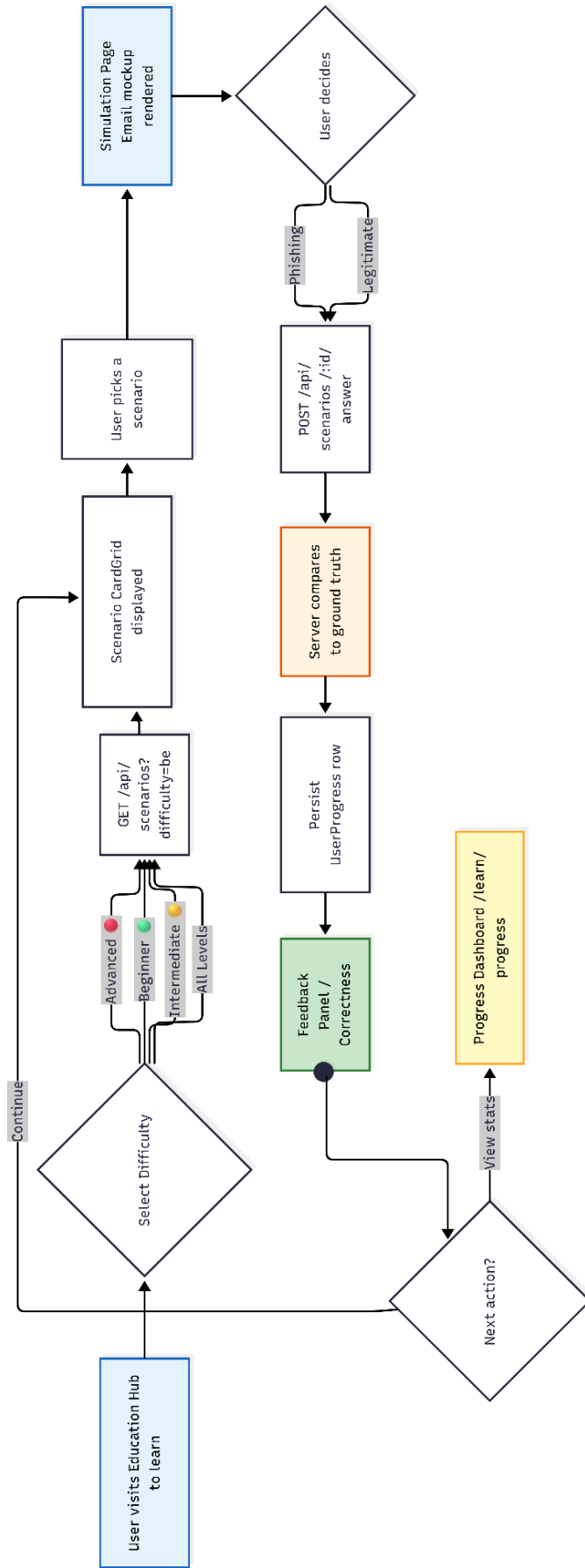
This design decision is important for two reasons. First, it separates the explanation from the classifier: because the flags are generated from the same engineered features the model saw, a user can always see which concrete signals contributed to a high-risk verdict, even though the model itself is a tree ensemble rather than a directly interpretable linear classifier. Second, the engine provides a sensible output even on messages the model considers low-risk, because the same heuristic checks run unconditionally; this means a user submitting a borderline email will still receive specific, actionable feedback rather than a bare 'low risk' verdict. The raw probability is then mapped into one of four user-facing verdict bands (high, elevated, low, minimal) using fixed thresholds, and the 0–100 risk score shown in the interface is a simple linear scaling of the probability. The bands are chosen to give a more informative display than a flat phishing-or-legitimate flip while remaining anchored to the model's underlying decision threshold of 0.5.

## 4.5 Educational Platform

The educational platform is implemented as a second Flask blueprint sharing the same application factory, database, and CORS configuration as the detection backend. Its data model consists of two SQLAlchemy entities. The Scenario table holds seeded training emails, each annotated with a difficulty level (beginner, intermediate, or advanced), a ground-truth label, a JSON-encoded list of phishing indicators, an explanation, and a list of learning points. The UserProgress table records one row per scenario attempt, linking a scenario identifier to an anonymous session identifier, the submitted answer, a correctness flag, time taken, and a completion timestamp.

User identity is handled through anonymous session identifiers generated client-side and stored in the browser's localStorage. This keeps the system free of any authentication machinery, which is appropriate for the project's academic scope and removes a substantial source of implementation complexity while still allowing per-user progress to be tracked across a browsing session. The trade-off is that progress does not persist across devices and cannot be recovered if local storage is cleared, which is accepted as a known limitation and recorded in Chapter 6.

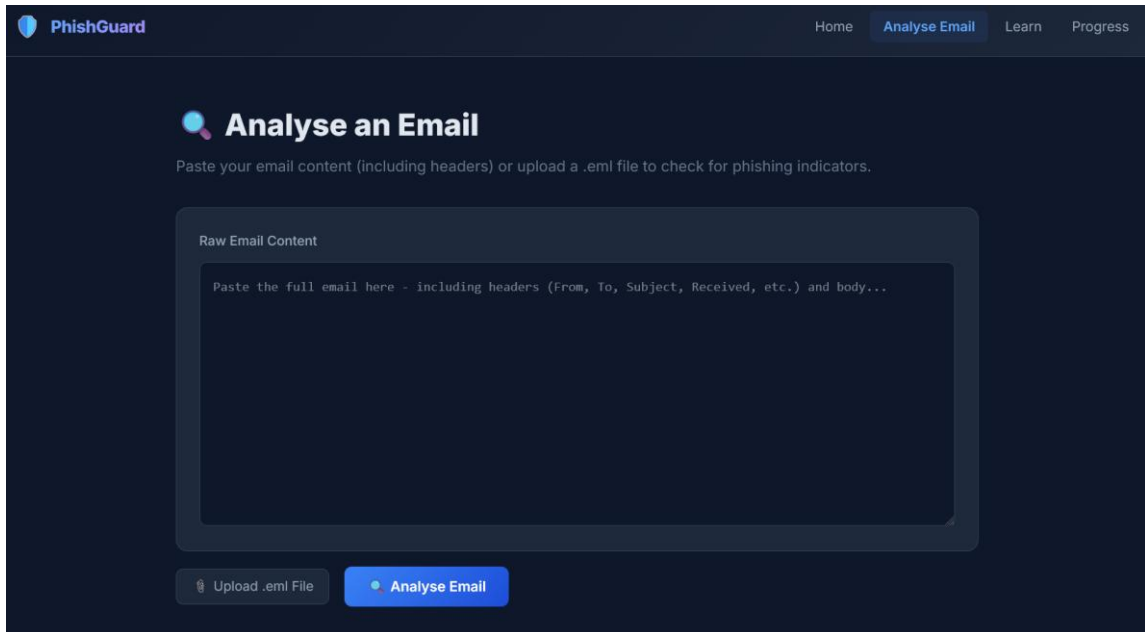
When a scenario is fetched through GET /api/scenarios/<id>, the Scenario.to\_dict() serialiser explicitly omits the is\_phishing, indicators, and explanation fields so that the correct answer cannot be inferred from network traffic before the user commits to a guess. Only after the user submits an answer via POST /api/scenarios/<id>/answer does the server compare the submission against the ground truth, persist a UserProgress row, and return the full explanation and learning points alongside a correctness flag. This client-server separation resembles the teaching structure of the simulation: the user must commit to an interpretation of the email before receiving formative feedback.



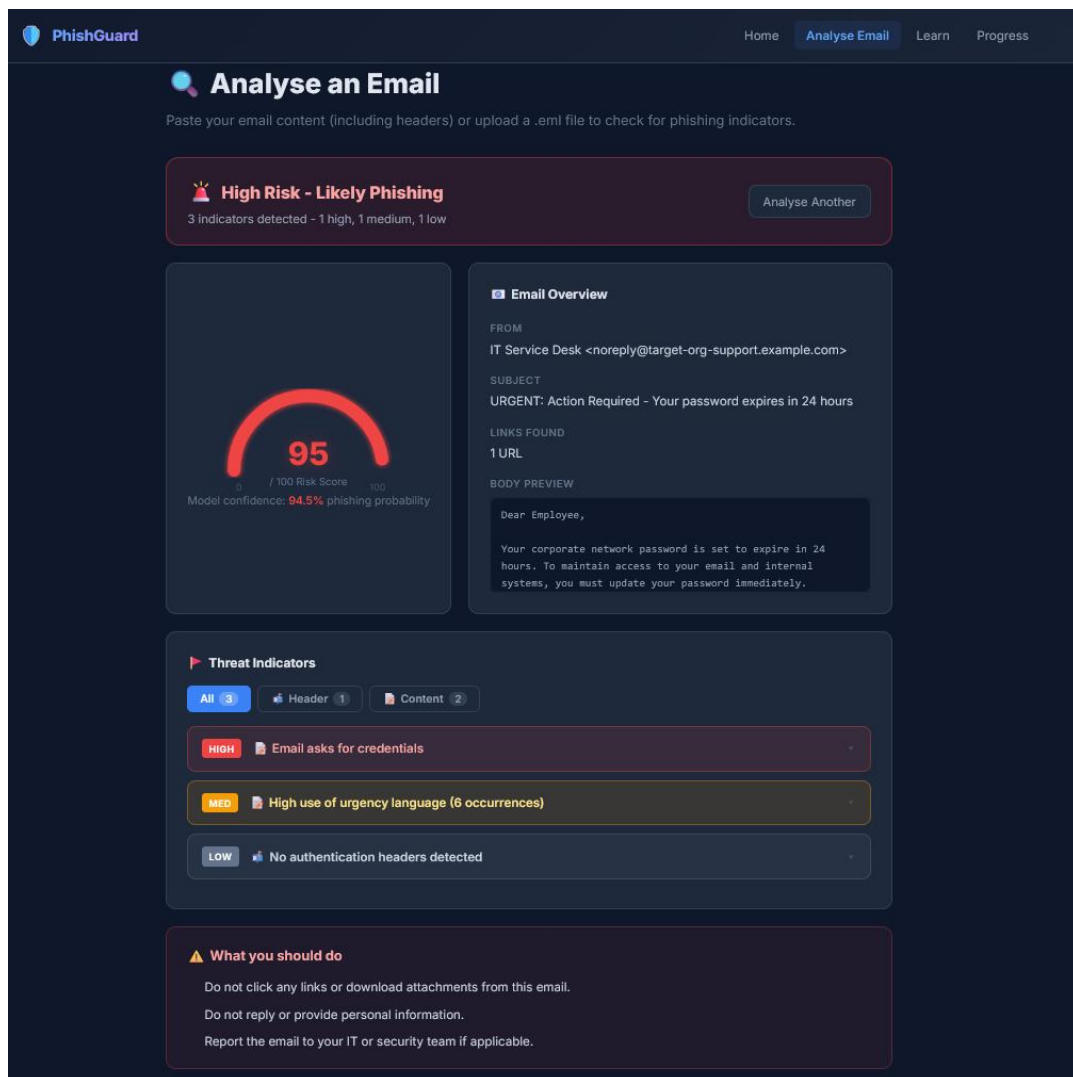
**Figure 4.3:** Educational platform user journey from scenario selection through feedback and progress review.

## 4.6 Frontend Architecture

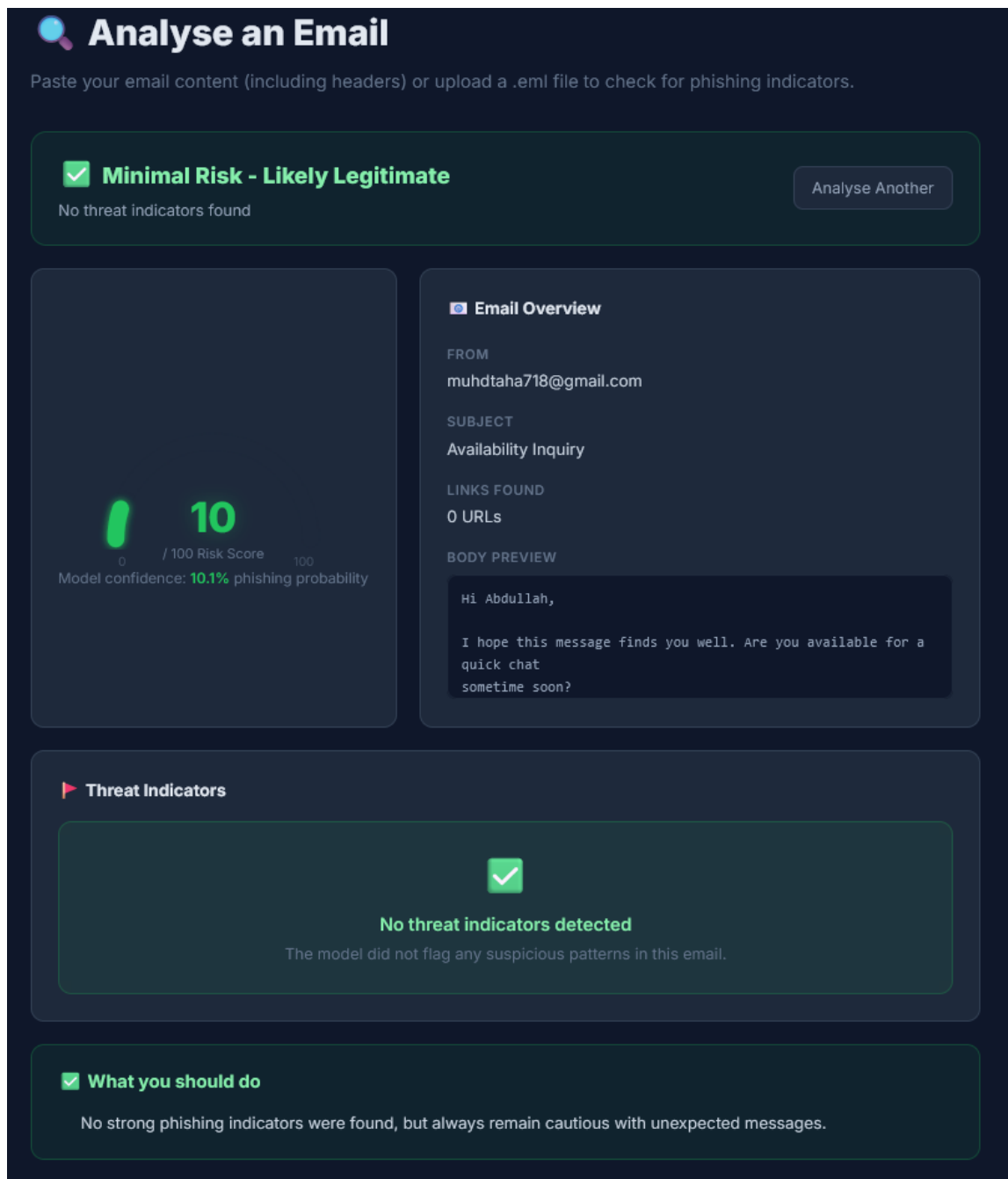
The user interface is a React 19 single-page application built with Vite and styled with Tailwind CSS v4 layered under a custom CSS design system. Routing between the four primary views: home, detection, education hub, and individual simulation. All of these are handled by react-router-dom. The detection page comprises an EmailInputForm component for paste-or-upload submission and a composite AnalysisResults component rendering a verdict banner, risk score gauge, threat indicators grouped by severity, and a summary recommendations panel. State is managed through a custom useAnalysis hook that isolates API call semantics from the page components.

The screenshot shows the 'Analyse an Email' page of the PhishGuard application. The page has a dark blue theme. At the top, there is a navigation bar with the PhishGuard logo on the left and links for 'Home', 'Analyse Email' (which is highlighted), 'Learn', and 'Progress' on the right. The main heading is 'Analyse an Email' with a magnifying glass icon. Below the heading, a subtitle reads: 'Paste your email content (including headers) or upload a .eml file to check for phishing indicators.' The central part of the page features a large, light blue rectangular box labeled 'Raw Email Content'. Inside this box, there is a text area with the placeholder text: 'Paste the full email here - including headers (From, To, Subject, Received, etc.) and body...'. At the bottom of the page, there are two buttons: a grey button labeled 'Upload .eml File' and a blue button labeled 'Analyse Email'.

*Figure 4.4: Detection page email input form, supporting both paste and .eml upload.*

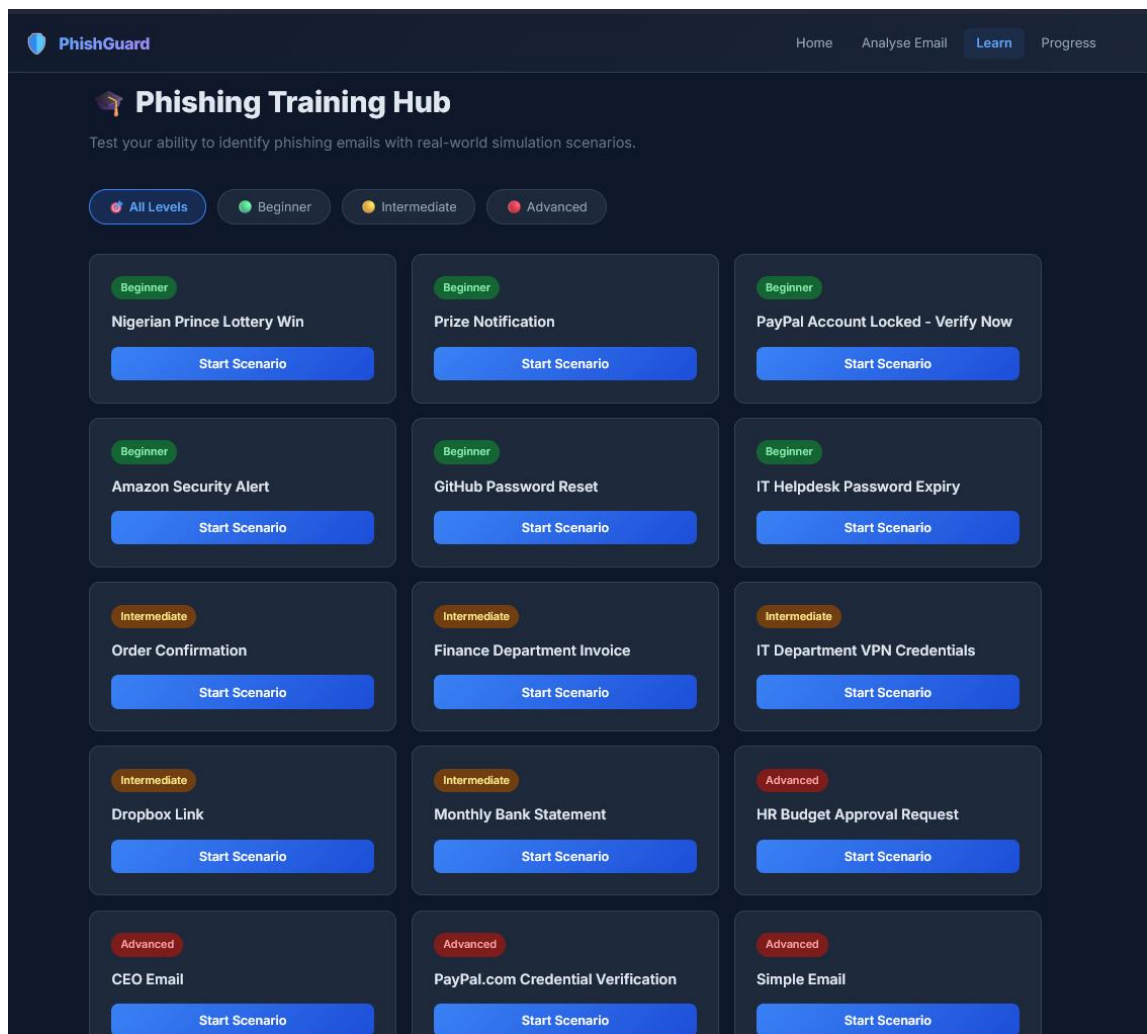


*Figure 4.5: Detection results view for a phishing email, showing risk score gauge, threat flags grouped by severity, and actionable recommendations.*

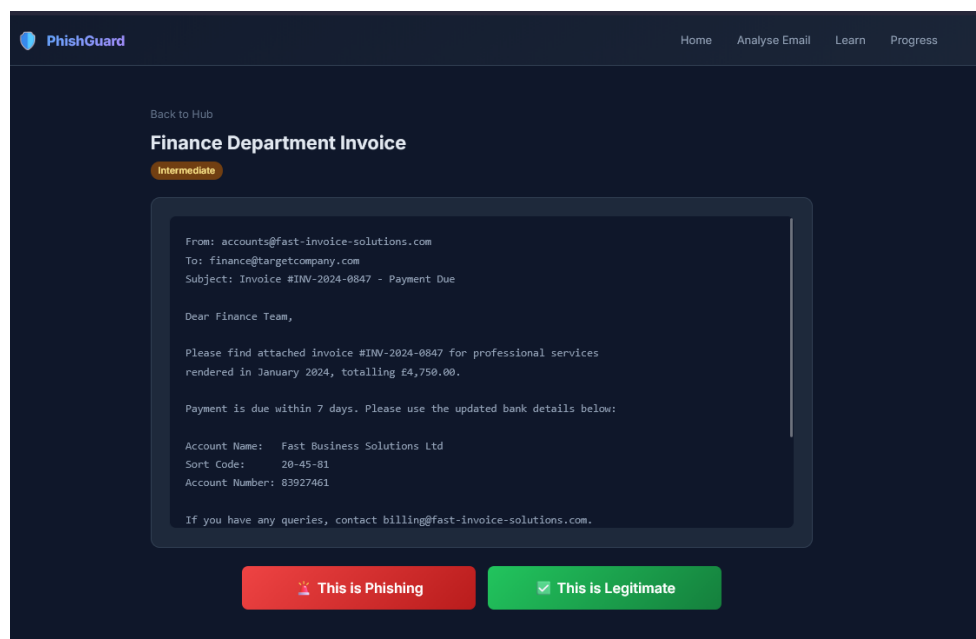


*Figure 4.6: Detection results view for a legitimate email, showing a minimal-risk verdict and the absence of phishing indicators.*

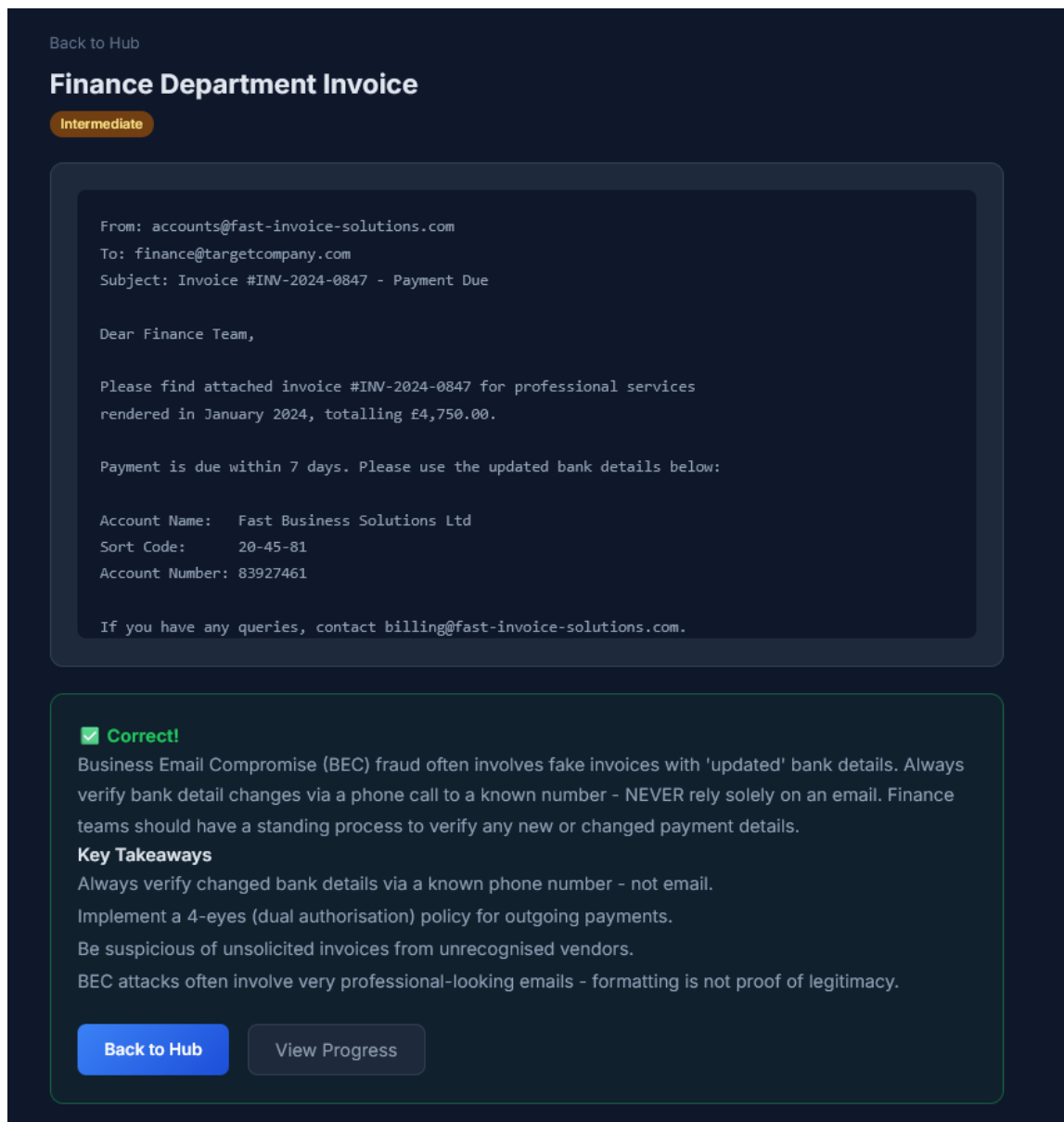




*Figure 4.7: Education Hub page with difficulty filter and grid of training scenarios.*



*Figure 4.8: Simulation page presenting a scenario email in a realistic email-client mockup with binary answer controls.*

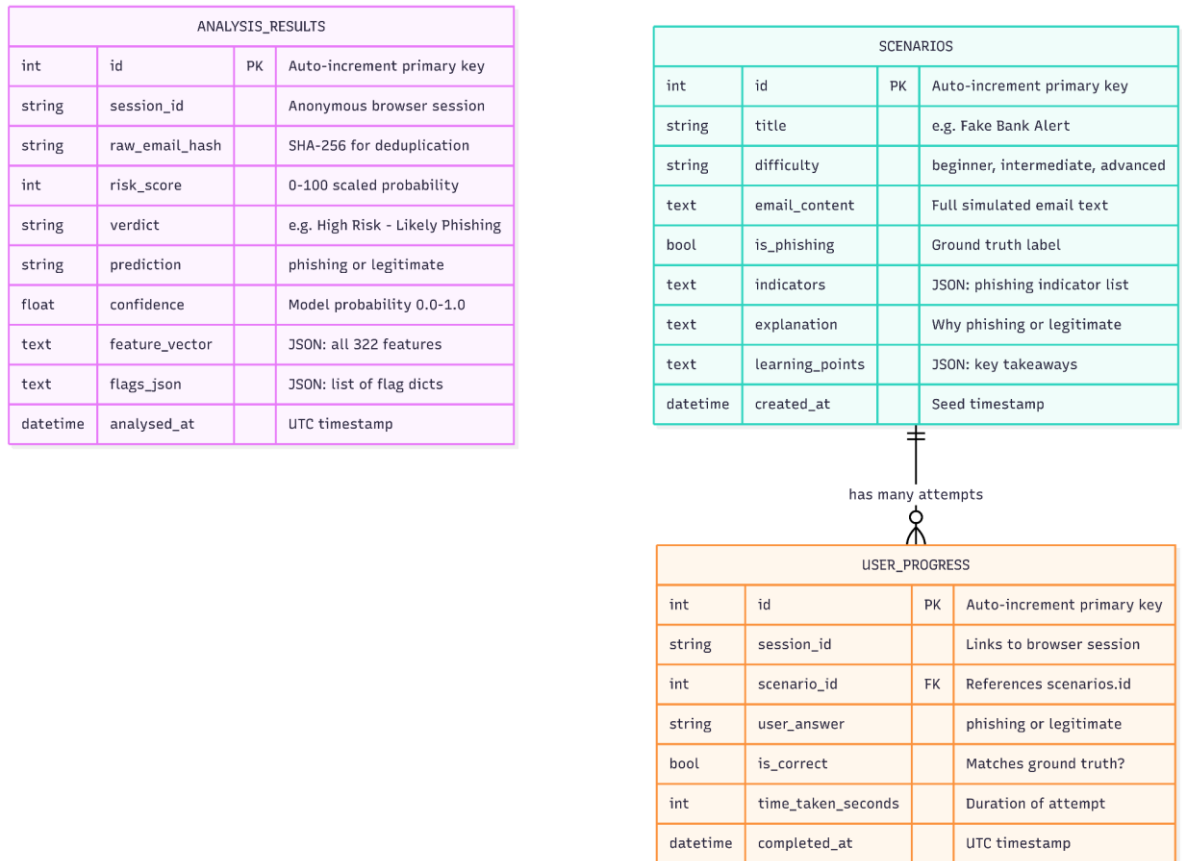


**Figure 4.9:** Feedback panel revealed after a scenario answer, showing correctness, detailed explanation, and key learning points.

The education hub presents a filterable grid of scenario cards. Selecting a scenario navigates to the simulation page, which renders the email in a realistic mockup with two answer buttons. The feedback panel then reveals the correct label, explanation, and learning points. The anonymous session identifier is generated client-side and reused across both detection and education flows.

## 4.7 Database Schema and Persistence

The schema consists of three SQLAlchemy tables: `analysis_results` (persists each detection submission), `scenarios` (seeded training emails for the educational simulation), and `user_progress` (records per-scenario attempts with a foreign key to `scenarios`). All tables are created via `db.create_all()` at startup. The `analysis_results` table stores feature vectors and flags as JSON blobs for schema simplicity, which is acceptable given that analytics over historical submissions is not an in-scope requirement.



**Figure 4.10:** Entity-Relationship diagram showing the three SQLite tables and their relationships. The solid line from `user_progress` to `scenarios` represents a foreign-key constraint. The `analysis_results` table is intentionally unrelated to the other tables, as detection submissions and educational interactions are independent workflows with no referential dependency.

The `analysis_results` table is intentionally unlinked to the `scenarios` and `user_progress` tables because detection submissions and educational interactions represent independent workflows with no referential dependency. A user may analyse emails without engaging with the educational platform, and vice versa. The `session_id` column in both `analysis_results` and `user_progress` is a client-generated identifier used for grouping within each workflow rather than a foreign key enforcing cross-workflow integrity.

## 5 Chapter 5: Tool Results and Evaluation

This chapter evaluates both components of the Phishing Detection and Awareness Tool against the objectives stated in Chapter 1, reporting model comparison, cross-validation stability, held-out test performance, per-source analysis, feature importance, and a heuristic evaluation of the educational platform.

### 5.1 Evaluation Setup

The evaluation uses the 27,180-email corpus assembled and audited in Section 3.2 (Table 3.5), comprising 4,897 phishing emails (18.0%) from Nazario\_5 and Nigerian\_Fraud and 22,283 legitimate emails (82.0%) from Enron, SpamAssassin, and Ling. The natural class ratio was preserved in all evaluation partitions.

The corpus was split 60/20/20 into training, validation, and test partitions with SMOTE applied to the training partition only, as detailed in Sections 3.4 and 3.5. Three classifiers were compared on the validation partition before hyperparameter tuning of the selected Random Forest via RandomizedSearchCV (Table 3.4). The tuned model was evaluated exactly once on the held-out test partition to produce the final metrics reported below.

### 5.2 Model Comparison

Table 5.1 compares the three classifiers on the validation partition:

*Table 5.1: Classifier comparison on the validation partition (5,436 emails).*

| Classifier           | Accuracy      | Precision     | Recall        | F1-score      | ROC-AUC       |
|----------------------|---------------|---------------|---------------|---------------|---------------|
| Logistic Regression  | 0.9862        | 0.9529        | 0.9714        | 0.9621        | 0.9981        |
| <b>Random Forest</b> | <b>0.9890</b> | <b>0.9801</b> | <b>0.9581</b> | <b>0.9690</b> | <b>0.9955</b> |
| XGBoost              | 0.9903        | 0.9773        | 0.9683        | 0.9728        | 0.9991        |

XGBoost achieved the highest F1-score (0.9728), followed by Random Forest (0.9690). However, the 0.38-percentage-point margin falls within the cross-validation standard deviation (0.0058). Random Forest was selected as the primary classifier because it provides native Gini importance scores that the explanation engine consumes directly, and because its simpler dependency profile better suits the Flask deployment architecture. The full justification is discussed in Section 6.1.

### 5.3 Cross-Validation Stability

To confirm that the observed validation metrics were not artefacts of a particular split, five-fold stratified cross-validation was conducted on the combined training and validation partitions (21,744 emails) using the tuned Random Forest configuration. The resulting per-fold F1-scores were 0.9543, 0.9687, 0.9634, 0.9683, and 0.9573, yielding a mean F1 of 0.9624 with a standard deviation of 0.0058. The narrow spread across folds indicates that the model's performance is stable and does not depend on any single

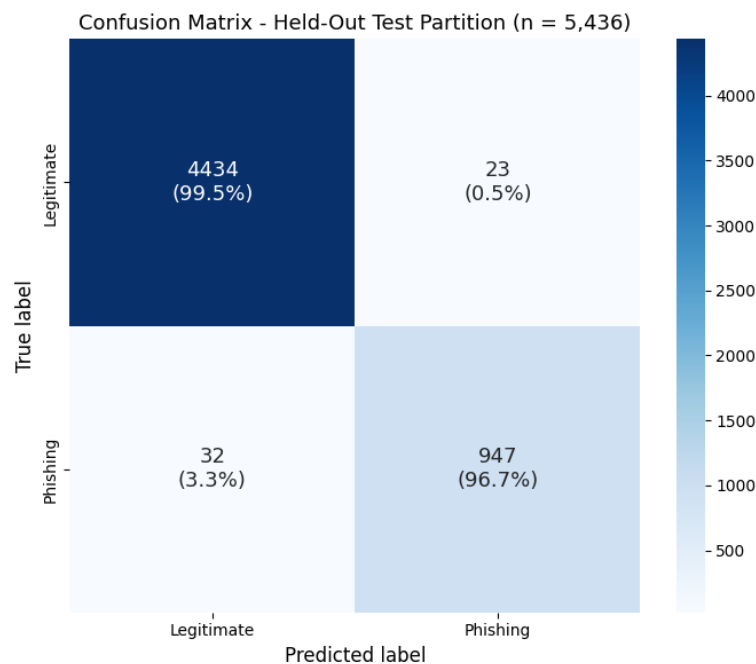
partition of the data. The close agreement between the cross-validation mean (0.9624), the single-split validation F1 (0.9690), and the subsequent held-out test F1 (0.9718, reported in Section 5.5) further suggests that the model generalises reliably within the distribution of the assembled corpus and shows no evidence of overfitting to the training partition.

## 5.4 Held-Out Test Performance

The tuned Random Forest was evaluated exactly once on the 5,436-email test partition. Table 5.2 reports the results, and Figure 5.1 shows the confusion matrix.

**Table 5.2:** Final held-out test metrics for the tuned Random Forest ( $n = 5,436$ ).

| Metric          | Value                    |
|-----------------|--------------------------|
| Accuracy        | 0.9899                   |
| Precision       | 0.9763                   |
| Recall          | 0.9673                   |
| F1-score        | 0.9718                   |
| ROC-AUC         | 0.9976                   |
| True positives  | 947                      |
| False positives | 23 (0.52% of legitimate) |
| False negatives | 32 (3.27% of phishing)   |
| True negatives  | 4,434                    |



**Figure 5.1:** Confusion matrix for the tuned Random Forest on the held-out test partition. Rows denote true labels; columns denote predicted labels.

The classifier achieves 97.18% F1 with 23 false positives and 32 false negatives. The asymmetry reflects a slight bias toward precision, which is desirable for a user-facing tool: over-flagging legitimate messages would erode user trust faster than occasionally missing phishing. The high ROC-AUC (0.9976) reflects the genuine separability of the two classes in this corpus; the per-source analysis in Section 5.5 demonstrates that performance nevertheless varies meaningfully across phishing sub-genres.

## 5.5 Per-Source Evaluation

Because the training corpus draws its positive class from two stylistically distinct phishing sub-genres: advance-fee fraud and credential phishing. A source-disaggregated evaluation was conducted to expose any distributional bias concealed by the aggregate metrics. Table 5.3 reports per-source accuracy for legitimate sources and per-source recall for phishing sources on the held-out test partition.

*Table 5.3: Per-source performance on the held-out test partition.*

| Source         | Class      | n     | Score           |
|----------------|------------|-------|-----------------|
| SpamAssassin   | Legitimate | 800   | 99.38% accuracy |
| Enron          | Legitimate | 3,117 | 99.55% accuracy |
| Ling           | Legitimate | 540   | 99.26% accuracy |
| Nigerian_Fraud | Phishing   | 676   | 98.52% recall   |
| Nazario_5      | Phishing   | 303   | 92.74% recall   |

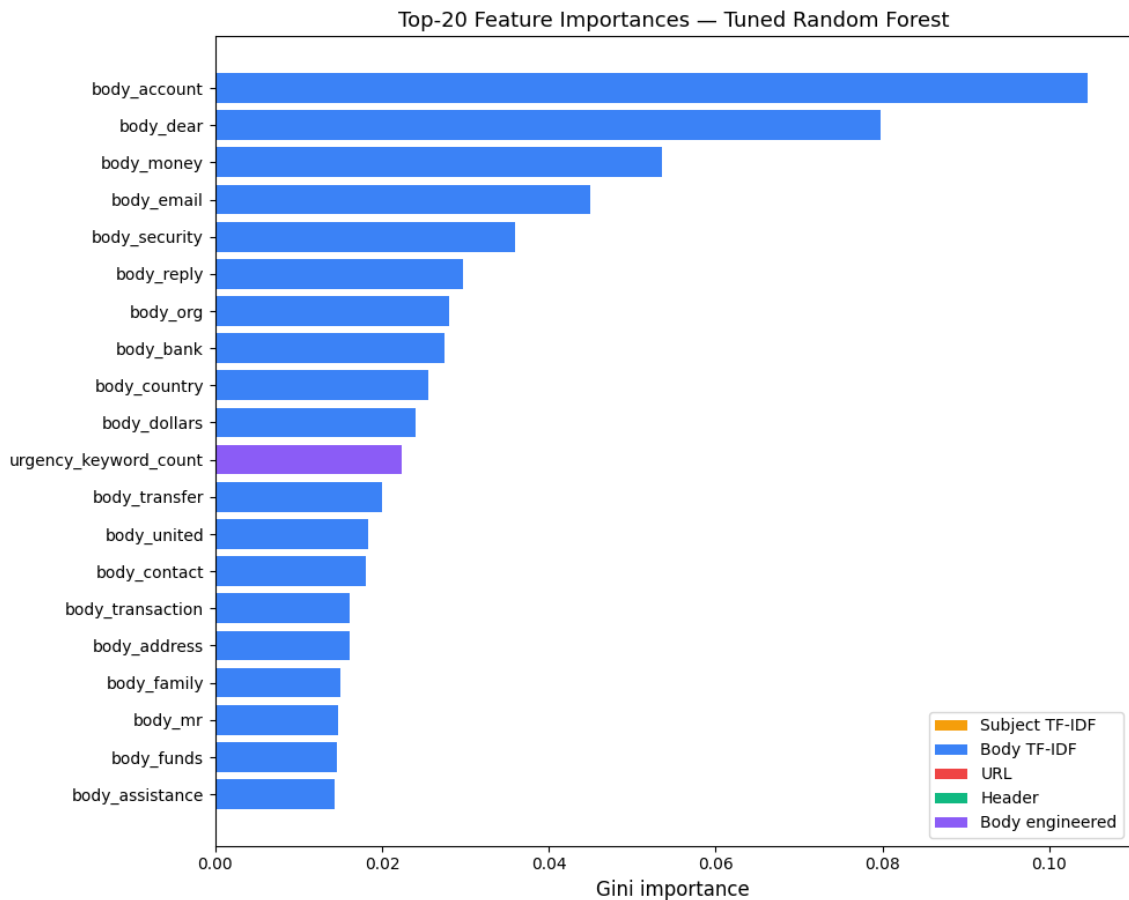
Two observations warrant comment. First, performance on all three legitimate sources is uniformly high, exceeding 99.2% accuracy on each, which confirms that the classifier is not exploiting any dataset-specific shortcut feature to distinguish legitimate from phishing emails. Second, recall on Nigerian fraud examples (98.52%) substantially exceeds recall on Nazario credential phishing (92.74%), a gap of 5.78 percentage points. This gap is a direct consequence of class composition: Nigerian\_Fraud contributes 68.0% of the phishing training examples and is dominated by distinctive advance-fee vocabulary, whereas Nazario contains the more modern and more operationally dangerous credential-harvesting subtype, for which the training signal is proportionally weaker.

This finding does not undermine the validity of the aggregate metrics; the classifier still exceeds 92% recall on the harder subtype, but it does constrain the claims that can be made about generalisation to contemporary credential phishing. The implications of this distributional bias for real-world deployment are analysed in Section 6.4, and a three-class extension that would disentangle advance-fee fraud, credential phishing, and legitimate email is proposed as future work in Section 7.2.

## 5.6 Feature Importance

Figure 5.2 reports the twenty most important features as ranked by the Random Forest's built-in Gini importance. The top-ranked features are dominated by body-level TF-IDF terms characteristic of advance-fee fraud vocabulary: `body_account`, `body_dear`, `body_money`, `body_email`, `body_security`, `body_dollars`, `body_bank`, `body_reply`, and `body_org`, alongside the engineered `urgency_keyword_count` feature. Header-derived

features rank substantially lower than either body-level TF-IDF features or engineered content features, confirming the limitation noted in Chapter 3 that inconsistent header preservation across the source corpora prevents the SPF, DKIM, and sender-mismatch signals from contributing meaningfully to the classifier's decisions.



**Figure 5.2:** Top-20 feature importances ranked by Random Forest Gini importance.

The dominance of advance-fee vocabulary in the feature ranking provides a mechanistic explanation for the per-source recall gap reported in Section 5.5 and is consistent with the observation by Saka et al. (2024) that classifiers trained on heterogeneous corpora reflect the vocabulary of the dominant sub-genre. The implications of this distributional bias are discussed in Section 6.2.

## 5.7 Operating Point and Threshold Analysis

The metrics above assume the default classification threshold of 0.5. For a security-first deployment, this threshold could be lowered to trade precision for recall. The prototype ships with the 0.5 default and exposes the raw probability in the API response, allowing a deploying operator to select an application-specific operating point without retraining.

## 5.8 Educational Platform Evaluation

Because no controlled user study was conducted within the project timeline, the educational platform was evaluated heuristically against the ten usability heuristics of Nielsen (1994), supplemented by a self-directed walkthrough that exercised all sixteen simulation scenarios across the three difficulty tiers. Table 5.4 summarises the findings.

**Table 5.4:** *Heuristic evaluation of the educational platform against Nielsen (1994).*

| Nielsen heuristic               | Status  | Observation                                                                              |
|---------------------------------|---------|------------------------------------------------------------------------------------------|
| Visibility of system status     | Pass    | Loading states and answer feedback are rendered immediately on interaction.              |
| Match with real world           | Pass    | Scenarios use plain language framed around familiar email contexts.                      |
| User control and freedom        | Pass    | Users can return to the hub from any scenario without losing session progress.           |
| Consistency and standards       | Pass    | Detection and education pages share a common layout, colour scheme, and navigation bar.  |
| Error prevention                | Pass    | Difficulty selector is a constrained control; free-text entry is avoided where possible. |
| Recognition over recall         | Pass    | Previous answers are visible in the progress dashboard during a session.                 |
| Flexibility and efficiency      | Partial | Difficulty selection does not persist across reloads (logged as future work).            |
| Aesthetic and minimalist design | Pass    | Interface avoids unnecessary chrome; scenario content dominates the viewport.            |
| Help users recognise errors     | Pass    | Incorrect answers reveal a detailed explanation and three learning points.               |
| Help and documentation          | Partial | Progress dashboard lacks a cumulative score across sessions (logged as future work).     |

During the walkthrough, all sixteen scenarios loaded and scored correctly, the feedback panel revealed the intended explanation and learning points for both correct and incorrect answers, and the anonymous session-based progress tracker correctly aggregated per-scenario outcomes across a simulated session. The evaluation surfaced two minor issues: the difficulty selector did not persist its state across page reloads, and the progress dashboard did not display a cumulative score aggregated across sessions. Both were logged for future work. A formal user study measuring pre- and post-training awareness gains, following the methodology established by Tschakert and Ngamsuriyaroj (2019) and refined by Ahmad et al. (2023), is proposed in Section 7.2 as the primary route to quantitative evaluation of the educational component.

### 5.8.1 Professional Evaluation from Experts

To supplement the heuristic evaluation, structured feedback was collected from four computing academics (three doctoral-level staff and one PhD researcher in computer science) who independently reviewed the complete system. Each evaluator interacted with both the detection engine and the educational platform before completing a seven-item

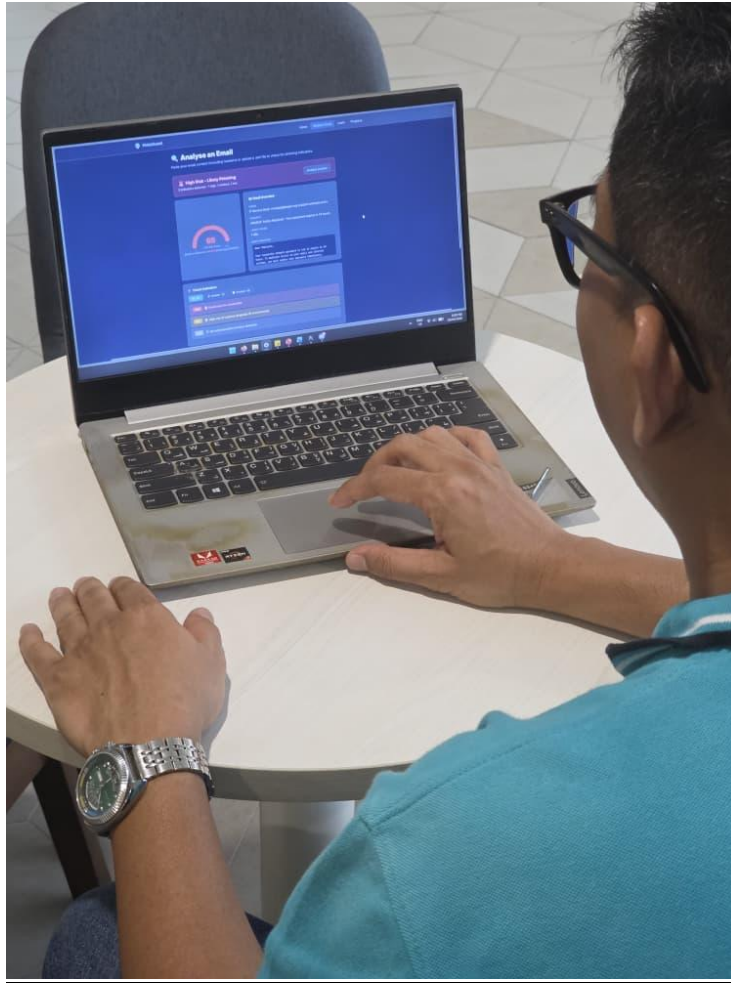


Likert-scale questionnaire (1 = strongly disagree, 5 = strongly agree) accompanied by an open-ended comment field. Table 5.5 summarises the quantitative ratings.

**Table 5.5:** Expert evaluation ratings (1–5 Likert scale,  $n = 4$ ).

| Criteria                           | Expert 1 | Expert 2 | Expert 3 | Expert 4 | Mean |
|------------------------------------|----------|----------|----------|----------|------|
| Detection accuracy                 | 4        | 4        | 4        | 3        | 3.75 |
| Risk score and explanation clarity | 5        | 4        | 4        | 4        | 4.25 |
| Scenario realism                   | 5        | 4        | 4        | 5        | 4.50 |
| Feedback effectiveness             | 5        | 4        | 5        | 5        | 4.75 |
| Interface usability                | 5        | 4        | 4        | 4        | 4.25 |
| Detection–education integration    | 5        | 4        | 4        | 5        | 4.50 |
| Usefulness for non-technical users | 5        | 4        | 4        | 5        | 4.50 |

All seven criteria received mean ratings at or above 3.75, with the highest scores awarded to feedback effectiveness (4.75) and scenario realism (4.50). Detection accuracy received the lowest mean (3.75); one evaluator noted that legitimate emails occasionally received elevated risk scores, an observation consistent with the false-positive rate reported in Section 5.4. Qualitative comments converged on three themes. First, evaluators endorsed the integrated detection-and-education design, describing the combination of automated analysis with an interactive training component as well-conceived. Second, two evaluators recommended expanding the training corpus to improve detection performance, echoing the dataset limitation analysed in Section 6.4. Third, evaluators suggested that a formal user study with non-technical participants and a more interactive, gamified educational experience would strengthen the tool, directions already identified as future work in Section 7.2.



*Figure 5.3: Expert user evaluating the detection component of the tool*

## 5.9 Summary

The tuned Random Forest classifier attains a 97.18% F1-score on the held-out test partition, with stable performance across five-fold cross-validation (mean F1 = 0.9624, standard deviation = 0.0058) and near-perfect class separation (ROC-AUC = 0.9976). Per-source analysis exposes a 5.78-point recall gap between advance-fee fraud and credential phishing, which is traceable to the composition of the training corpus and is reflected directly in the feature-importance ranking. The educational platform passes a heuristic evaluation against Nielsen's ten principles, with two minor usability issues logged for future iteration. Chapter 6 interprets these findings critically, examines the threats to validity that constrain the claims drawn from them, and situates the distributional bias within the broader methodological context of public phishing email research.

## 6 Chapter 6: Analysis and Reflection

This chapter critically interprets the results of Chapter 5, situates them within the constraints of the available data and chosen methodology, and identifies the threats to validity that limit the claims the project can make.

### 6.1 Interpreting System Outcomes

The 97.18% F1-score is competitive with the classical ML results reported by Adeyemi Onih (2024) and Naz et al. (2024), though it falls short of the transformer-based accuracies of Alhuzali et al. (2025), whose BERT and RoBERTa models reached 99.08% on a merged balanced corpus. Direct numerical comparison is inappropriate, however, because the two studies evaluate different prediction targets: Alhuzali et al. treat commercial spam and phishing as a single positive class, whereas the present work restricts the positive class to genuine social-engineering phishing. The consistency across evaluation partitions, which displays a validation F1 of 0.9690, a cross-validation mean of 0.9624 ( $\sigma = 0.0058$ ), and a held-out test F1 of 0.9718, is the most important robustness signal. The narrow spread confirms that no overfitting occurred, a direct consequence of the single-evaluation discipline applied to the test partition (Section 3.5). The selection of Random Forest over the marginally higher-performing XGBoost (Section 5.2) traded 0.38 percentage points of F1 for native Gini importance scores and a simpler deployment profile. This is a trade-off that appears sound for the project's academic scope but would warrant revisiting under stricter production requirements.

### 6.2 Misclassification Analysis and Per-Source Bias

The per-source recall gap reported in Section 5.5 carries a significant safety implication: credential phishing is the operationally more dangerous subtype and the dominant contemporary threat (Jabir et al., 2025), yet the classifier's discriminative power is concentrated on the historically easier advance-fee fraud problem, as the feature-importance analysis in Section 5.6 confirms. The 92.74% recall on credential phishing is not poor in absolute terms, but is meaningfully lower than the 96.73% aggregate recall would suggest. The false-positive rate on legitimate email (0.52%) is low enough that user-trust degradation would be minimal, and the default 0.5 threshold is defensible for the project's academic scope.

### 6.3 Technical Constraints

Several engineering limitations constrain the prototype. The header features (SPF, DKIM, Received-hop count) underperform because the source corpora stored raw headers inconsistently; a deployment connected to a live mail server would derive substantially more value from these signals. The TF-IDF dimensionality was capped at 300 terms as a pragmatic compromise between coverage and overfitting risk. The RandomizedSearchCV tuning was limited to ten iterations with three-fold cross-validation due to Google Colab compute constraints; a production pipeline would benefit from Bayesian optimisation and a larger fold count. The educational platform was evaluated heuristically rather than empirically, as no controlled user study was conducted within the project timeline, and a formal pre/post intervention study following Tschakert and Ngamsuriyaroj (2019) is proposed as future work in Chapter 7.

## 6.4 Reliability of Findings and Confounding Factors

Internal validity is supported by consistency across cross-validation folds and the single-evaluation discipline applied to the test set. The principal residual threat is dependence on SMOTE: synthetic minority samples may exploit feature-space regions that do not correspond to real phishing emails. Applying SMOTE only to the training partition rules out the most severe form of this threat, but an ablation retraining without SMOTE was not performed. Construct validity is the category in which this project has made its most deliberate investment. By auditing and rejecting corpora whose positive class conflates spam with phishing, the work ensures that the label being predicted corresponds to social-engineering phishing. However, the construct remains bounded by the two accepted corpora: genuine phishing is operationalised as examples from Nazario\_5 or Nigerian\_Fraud, which may not represent the full diversity of contemporary tactics.

External validity is the most significant limitation. The training corpus reflects phishing tactics from approximately 2005 to 2017. Credential phishing has since shifted toward LLM-generated spear-phishing templates and mobile-first attack surfaces, none of which are represented. Jabir et al. (2025) document this shift, presenting SpearBot (Qi et al., 2025) as evidence that LLM-generated phishing can evade detectors trained on pre-LLM data. The present work's claims are therefore limited to the class of phishing represented in the source corpora. Statistical conclusion validity is well supported for the aggregate metrics (5,436 test samples) but less so for per-source metrics: the Nazario recall estimate rests on only 303 samples, giving a standard error of roughly 1.5 percentage points. The qualitative finding that recall is lower on Nazario than on Nigerian\_Fraud is robust, but the precise gap should not be over-interpreted.

## 7 Chapter 7: Summary, Conclusions, and Future Directions

This chapter summarises the project's contributions, assesses the extent to which its objectives have been met, and identifies directions for future work. Section 7.1 revisits the four contributions claimed in Chapter 1 and evaluates each against the results reported in Chapter 5. Section 7.2 proposes six concrete directions for extending the work, grouped into methodological improvements, dataset expansions, architectural extensions, and empirical evaluations that the present project did not undertake.

### 7.1 Summary of Contributions

The project has designed, implemented, and evaluated a dual-function web application combining automated phishing detection (97.18% F1, 0.9976 ROC-AUC on held-out test) with interactive user education (sixteen scenarios across three difficulty tiers, passing heuristic evaluation against Nielsen's ten principles). The four contributions claimed in Chapter 1 are the dataset audit, per-source recall analysis, rule-based explanation engine, and integrated detection-education architecture. These are all supported by the evaluation in Chapter 5. All four project objectives have been met, with the acknowledged limitation that the educational platform's effectiveness was assessed heuristically rather than through a controlled user study.

### 7.2 Future Work

Six directions follow from the limitations analysed in Chapter 6. First, a modern phishing corpus is needed: the Monkey.org mbox archives (~744 credential-phishing emails with full headers) would provide an immediate out-of-distribution evaluation set, and a post-2020 corpus from donated inboxes would substantially improve external validity. Second, extending the classification target to three classes (phishing, spam, legitimate) would convert the rejected corpora from a methodological liability into a training asset. Third, a formal pre/post intervention study following Tschakert and Ngamsuriyaraj (2019) would provide the first quantitative measurement of the educational platform's educational effectiveness. Fourth, post-hoc probability calibration (Platt scaling or isotonic regression) would convert the Random Forest's output into statistically meaningful confidence values, enabling threshold-based routing in deployment. Fifth, replacing the singleton model with an MLOps retraining pipeline would allow periodic incorporation of user-submitted emails with confirmed labels. Sixth, evaluating whether features sensitive to LLM artefacts (perplexity, burstiness) provide a discriminative signal on AI-generated phishing would position the tool to track the evolving threat landscape documented by Jabir et al. (2025).

## References

- Adeyemi Onih, V. (2024) 'Phishing detection using machine learning: a model development and integration', *International Journal of Scientific and Management Research*, 7(4), pp. 27–63. doi: 10.37502/IJSMR.2024.7403.
- Ahmad, B.M., Ahmed, S.M. and Sylvanus, D.E. (2023) 'Enhancing phishing awareness strategy through embedded learning tools: a simulation approach', *Archives of Advanced Engineering Science*, pp. 1–14. doi: 10.47852/bonviewAAES32021392.
- Aldakheel, E.A., Razaque, A., Alotaibi, B., Alotaibi, M., Hariri, S. and Almiani, M. (2023) 'A deep learning-based innovative technique for phishing detection in modern security with uniform resource locators', *Sensors*, 23(9), 4403. doi: 10.3390/s23094403.
- Alhuzali, A., Alloqmani, A., Aljabri, M. and Alharbi, F. (2025) 'In-depth analysis of phishing email detection: evaluating the performance of machine learning and deep learning models across multiple datasets', *Applied Sciences*, 15(6), 3396. doi: 10.3390/app15063396.
- Aljofey, A., Jiang, Q., Qu, Q., Huang, M. and Niyigena, J.P. (2020) 'An effective phishing detection model based on character level convolutional neural network from URL', *Electronics*, 9(1), 180. doi: 10.3390/electronics9010180.
- Alluqmani, K., Karrar, A.E., Alhaidari, M., Alharbi, R. and Alharbi, S. (2025) 'Assessing the efficacy of security awareness training in mitigating phishing attacks: a review', *International Journal of Advanced Trends in Computer Science and Engineering*, 14(3), pp. 177–184. doi: 10.30534/ijatcse/2025/081432025.
- Androutsopoulos, I., Koutsias, J., Chandrinos, K.V., Paliouras, G. and Spyropoulos, C.D. (2000) 'An evaluation of naive Bayesian anti-spam filtering', in *Proceedings of the Workshop on Machine Learning in the New Information Age*, 11th European Conference on Machine Learning, Barcelona, Spain, pp. 9–17.
- Anti-Phishing Working Group (2022) *Phishing Activity Trends Report, 4th Quarter 2022*. Available at: <https://apwg.org/trendsreports/> (Accessed: 15 March 2026).
- Apache Software Foundation (2006) *SpamAssassin Public Corpus*. Available at: <https://spamassassin.apache.org/old/publiccorpus/> (Accessed: 10 January 2026).
- Breiman, L. (2001) 'Random forests', *Machine Learning*, 45(1), pp. 5–32. doi: 10.1023/A:1010933404324.
- Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P. (2002) 'SMOTE: synthetic minority over-sampling technique', *Journal of Artificial Intelligence Research*, 16, pp. 321–357. doi: 10.1613/jair.953.
- Chen, T. and Guestrin, C. (2016) 'XGBoost: a scalable tree boosting system', in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco, CA, 13–17 August. New York: ACM, pp. 785–794. doi: 10.1145/2939672.2939785.
- Jabir, R., Le, J. and Nguyen, C. (2025) 'Phishing attacks in the age of generative artificial intelligence: a systematic review of human factors', *AI*, 6(8), 174. doi: 10.3390/ai6080174.
- Metsis, V., Androutsopoulos, I. and Paliouras, G. (2006) 'Spam filtering with naive Bayes - which naive Bayes?', in *Proceedings of the Third Conference on Email and Anti-Spam (CEAS 2006)*, Mountain View, CA, 27–28 July, pp. 28–69.

- Naz, S., Yousaf, A., Rashid, M., Iqbal, J. and Shahzad, K. (2024) 'A comprehensive survey on social engineering-based attacks on social networks', *ICT Express*, 10(2), pp. 446–464. doi: 10.1016/j.icte.2024.01.004.
- Nazario, J. (2019) *Phishing Corpus*. Available at: <https://monkey.org/~jose/phishing/> (Accessed: 15 October 2025).
- Nielsen, J. (1994) *Usability Engineering*. San Francisco: Morgan Kaufmann.
- Pohan, F.J.R., Suherdi, D. and Iskandar, Y.S. (2023) 'Simulation and detection of phishing attacks on student academic emails using social engineering techniques', *International Journal of Health, Engineering and Technology*, 2(4), pp. 164–171.
- Qi, Q., Luo, Y., Xu, Y., Guo, W. and Fang, Y. (2025) 'SpearBot: leveraging large language models in a generative-critique framework for spear-phishing email generation', *Information Fusion*, 122, 103176. doi: 10.1016/j.inffus.2025.103176.
- Radev, D. (2008) *CLAIR Collection of Fraud Email (Nigerian Fraud)*. Ann Arbor, MI: University of Michigan. Available at: <https://hdl.handle.net/2027.42/115354> (Accessed: 15 October 2025).
- Rizzoni, F., Magalini, S., Casaroli, A., Mari, P., Dixon, M. and Coventry, L. (2022) 'Phishing simulation exercise in a large hospital: a case study', *Digital Health*, 8, pp. 1–13. doi: 10.1177/20552076221081716.
- Saka, T., Jain, R., Vaniea, K. and Kökciyan, N. (2024) 'Phishing codebook: a structured framework for the characterization of phishing emails', *arXiv preprint*, arXiv:2408.08967. Available at: <https://arxiv.org/abs/2408.08967> (Accessed: 10 December 2025).
- Tschakert, K.F. and Ngamsuriyaroj, S. (2019) 'Effectiveness of and user preferences for security awareness training methodologies', *Heliyon*, 5(6), e02010. doi: 10.1016/j.heliyon.2019.e02010.

## Appendix A.

### Agreed Project Brief

COMP3200-FYP (UoSM) 2025/26

|                      |                                       |
|----------------------|---------------------------------------|
| Student Name         | Abdullah Hashmat Azeez                |
| Agreed Project Title | Phishing Detection and Awareness Tool |
| Supervisor Name      | Dr. Madni                             |

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Project Purpose        | <p>This project addresses the challenge of email-based phishing attacks, which exploit human vulnerabilities via social engineering. The goal is to develop a dual-function platform that:</p> <ul style="list-style-type: none"><li>• Provides a technical solution to <b>detect</b> potential phishing attempts and privacy risks by analyzing email content</li><li>• Serves as a security <b>awareness</b> tool. This educational component will attempt to use interactive simulations to help users understand and identify social engineering tactics, data privacy concerns, and other email-based threats.</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Project Scope          | <ul style="list-style-type: none"><li>• <b>In Scope:</b><ul style="list-style-type: none"><li>○ <b>Detection Engine:</b> Development of a system to analyze email content (e.g., headers, body text, URLs) for phishing indicators. The system will likely analyze user-submitted content (pasted or uploaded emails) rather than integrating directly with a live email client.</li><li>○ <b>Educational Platform:</b> Creation of an interactive module to help train users to identify phishing attempts.</li><li>○ <b>User Interface (UI):</b> A user-friendly web-based interface that displays the analysis results, provides clear explanations for the detection (e.g., "This link is suspicious because..."), and offers actionable security recommendations.</li><li>○ <b>Deliverables:</b> The project will result in a working software prototype and the final project report, as per the FYP guidelines.</li></ul></li><li>• <b>Out of Scope:</b><ul style="list-style-type: none"><li>○ This project is not a real-time, commercial-grade email client plugin (e.g., for Gmail/Outlook).</li><li>○ The system will not automatically block or quarantine emails from a live server.</li><li>○ The detection focus is on phishing and email-based social engineering, not a comprehensive anti-virus or general anti-malware suite.</li></ul></li></ul> |
| Objectives (Tentative) | <ul style="list-style-type: none"><li>• To develop a system that can analyze email content and identify potential phishing attempts and privacy risks.</li><li>• To create an educational platform that simulates common phishing scenarios and helps users understand the risks associated with malicious emails.</li><li>• To design a user-friendly interface that provides actionable security recommendations, explanations of detection reasoning, and guidance on countermeasures.</li><li>• To test and evaluate the system's effectiveness in both detection accuracy and its ability to improve user security awareness.</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |



Signature of the Student

A handwritten signature in white ink on a dark background, appearing to be 'A. J. 2025'.

Date: 23/10/2025

Signature of the Supervisor

A handwritten signature in white ink on a dark background, appearing to be 'H. J. 2025'.

## Appendix B.

### Gantt Chart (Planned Schedule):

| Task                                  | Oct-25    | Nov-25 | Dec-25  | Jan-26        | Feb-26 | Mar-26 | Apr-26 | May-26 |
|---------------------------------------|-----------|--------|---------|---------------|--------|--------|--------|--------|
| Project Brief & Planning              |           |        |         |               |        |        |        |        |
| Literature Review                     |           |        |         |               |        |        |        |        |
| Technology Stack Research             |           |        |         |               |        |        |        |        |
| Progress Report                       |           |        |         |               |        |        |        |        |
| Dataset Collection & Preparation      |           |        |         |               |        |        |        |        |
| Detection Engine - Feature Extraction |           |        |         |               |        |        |        |        |
| Detection Engine - ML Model Training  |           |        |         |               |        |        |        |        |
| Educational Platform - Design         |           |        |         |               |        |        |        |        |
| Educational Platform - Implementation |           |        |         |               |        |        |        |        |
| Frontend/UI Development               |           |        |         |               |        |        |        |        |
| System Integration                    |           |        |         |               |        |        |        |        |
| Testing & Evaluation                  |           |        |         |               |        |        |        |        |
| Final Report Writing                  |           |        |         |               |        |        |        |        |
| Viva Preparation & Poster             |           |        |         |               |        |        |        |        |
|                                       | Completed |        | Planned | Current Month |        |        |        |        |

Gantt Chart (Actual Progress):

| Task                                  | Oct-25 | Nov-25    | Dec-25  | Jan-26        | Feb-26 | Mar-26 | Apr-26 | May-26 |
|---------------------------------------|--------|-----------|---------|---------------|--------|--------|--------|--------|
| Project Brief & Planning              |        |           |         |               |        |        |        |        |
| Literature Review                     |        |           |         |               |        |        |        |        |
| Technology Stack Research             |        |           |         |               |        |        |        |        |
| Progress Report                       |        |           |         |               |        |        |        |        |
| Dataset Collection & Preparation      |        |           |         |               |        |        |        |        |
| Detection Engine - Feature Extraction |        |           |         |               |        |        |        |        |
| Detection Engine - ML Model Training  |        |           |         |               |        |        |        |        |
| Educational Platform - Design         |        |           |         |               |        |        |        |        |
| Educational Platform - Implementation |        |           |         |               |        |        |        |        |
| Frontend/UI Development               |        |           |         |               |        |        |        |        |
| System Integration                    |        |           |         |               |        |        |        |        |
| Testing & Evaluation                  |        |           |         |               |        |        |        |        |
| Final Report Writing                  |        |           |         |               |        |        |        |        |
| Viva Preparation & Poster             |        |           |         |               |        |        |        |        |
|                                       |        | Completed | Planned | Current Month |        |        |        |        |

## Appendix C.

### User Manual:

PhishGuard is a web-based platform that combines machine-learning email analysis with interactive training scenarios to help users identify and defend against phishing attacks.

The tool serves two purposes:

1. **Detection** – Users paste raw email content (or upload `.eml`` files) and receive an instant risk assessment with human-readable explanations of every threat indicator found.
2. **Education** – Users work through curated phishing and legitimate email scenarios at varying difficulty levels, building practical detection skills with immediate feedback.

#### ## Prerequisites

- Python 3.10+
- Node.js 18+
- npm 9+

#### ## Setup

##### ### Backend

```
```bash
cd backend
python -m venv venv
venv\Scripts\activate      # Windows
# source venv/bin/activate  # macOS / Linux

pip install -r requirements.txt
```
```

Copy the environment file and adjust values as needed:

```
```bash
cp .env.example .env
```
```

The ML model files (`.pkl``) must be placed in `backend/app/ml/models/``. Four files are required:

- `phishing_model.pkl`` – trained Random Forest classifier
- `tfidf_subject.pkl`` – subject line TF-IDF vectoriser
- `tfidf_body.pkl`` – body text TF-IDF vectoriser
- `feature_names.pkl`` – ordered feature column names

These are produced by the training notebook and are not included in version control due to file size.

### ### Frontend

```
```bash
cd frontend
npm install
```
```

### ## Running the Application

Start the backend:

```
```bash
cd backend
python run.py
```
```

In a separate terminal, start the frontend:

```
```bash
cd frontend
npm run dev
```
```

The frontend runs at `http://localhost:5173` and proxies API requests to the Flask backend on port 5000.

### ### Seeding the Database

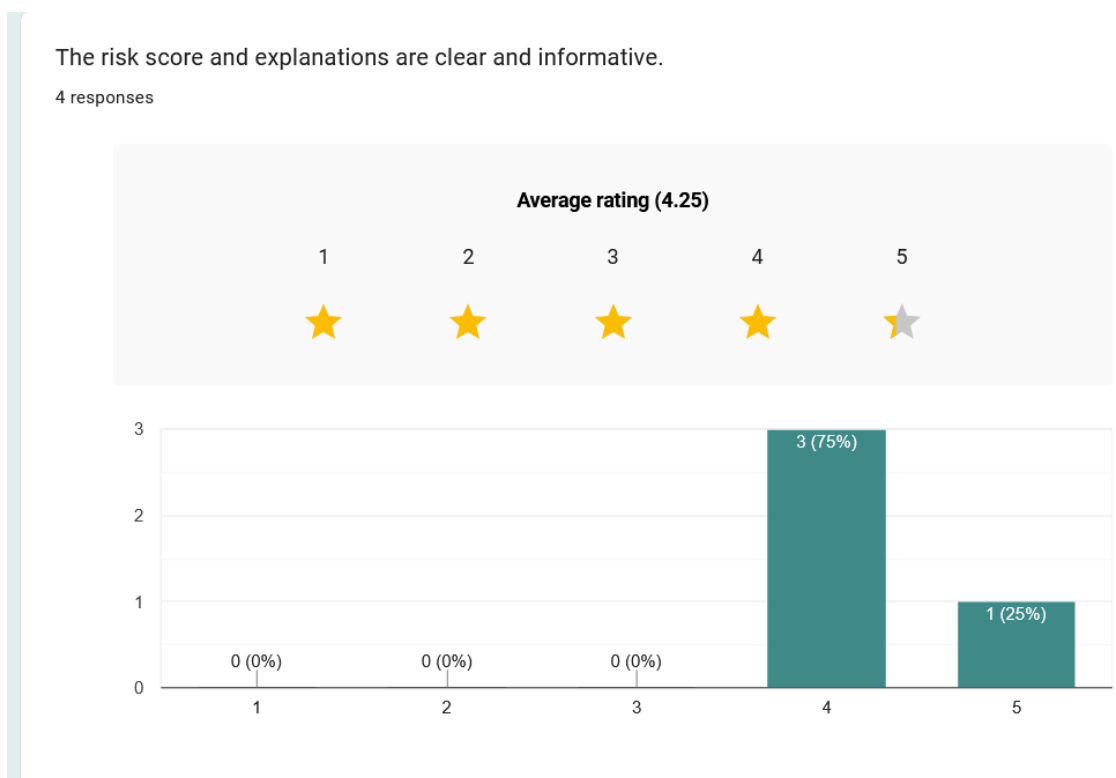
The education module requires training scenarios to be seeded into the database:

```
```bash
cd backend
python seed_scenarios.py
```
```

This populates the database with phishing and legitimate email scenarios across beginner, intermediate, and advanced difficulty levels. Run with `--force` to clear and re-seed.

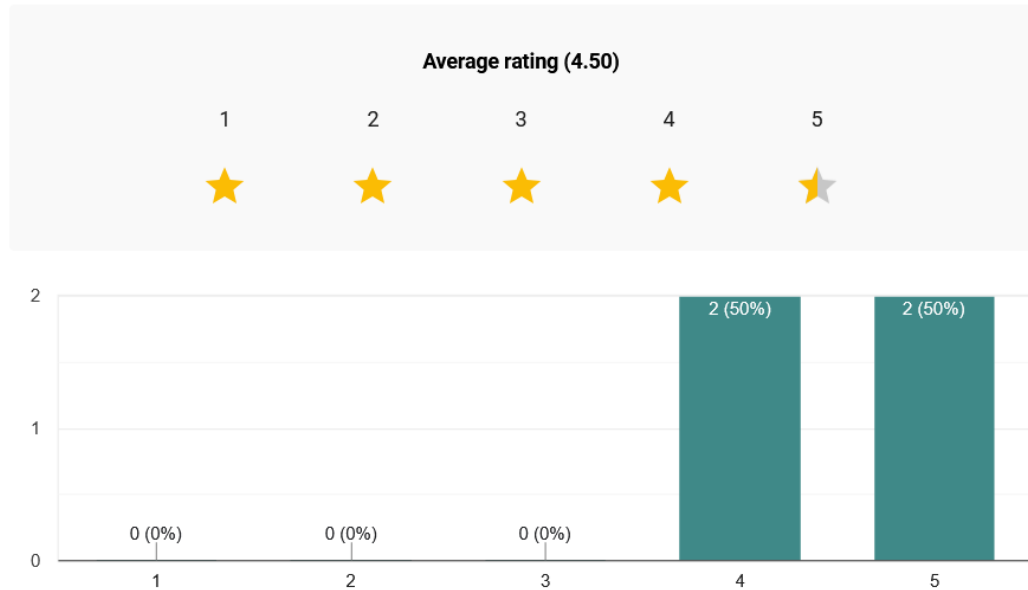
## Appendix D.

The following figures reproduce the response distributions from the expert evaluation questionnaire administered via Google Forms to four evaluators. The questionnaire used a five-point Likert scale (1 = Strongly Disagree, 5 = Strongly Agree) across seven criteria.



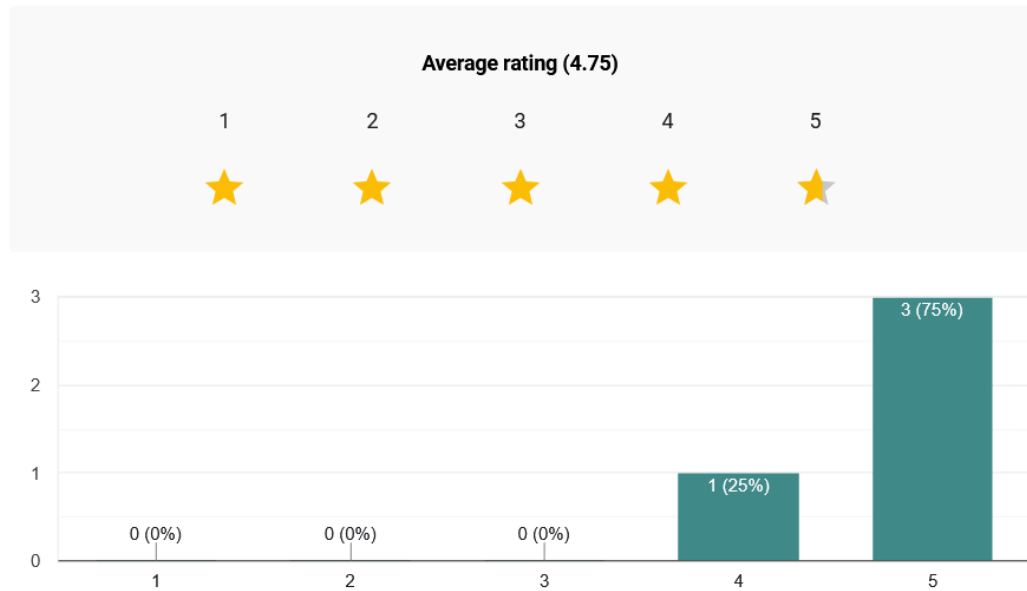
The educational scenarios are realistic and promote learning.

4 responses



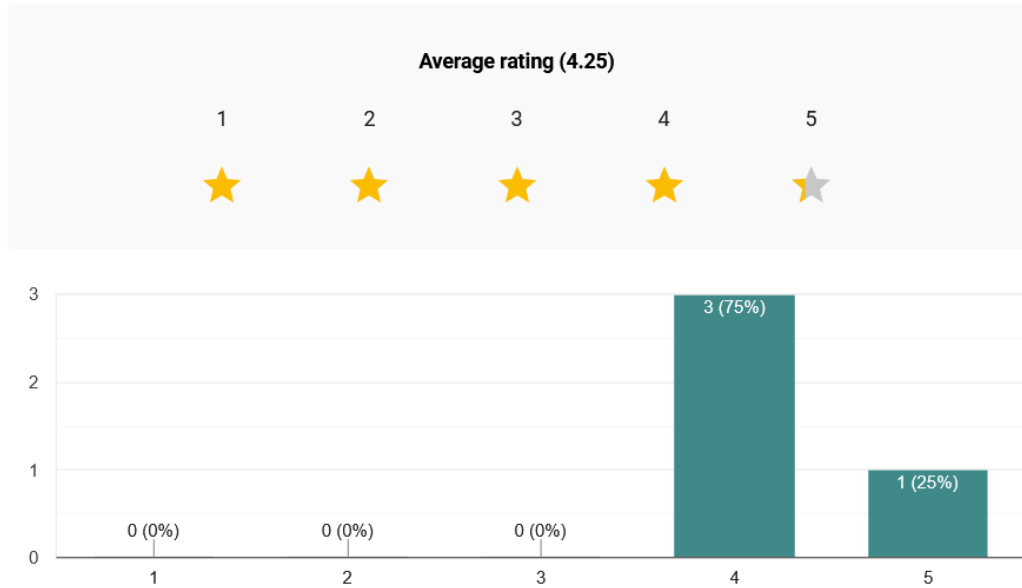
The feedback after each scenario effectively reinforces key concepts.

4 responses



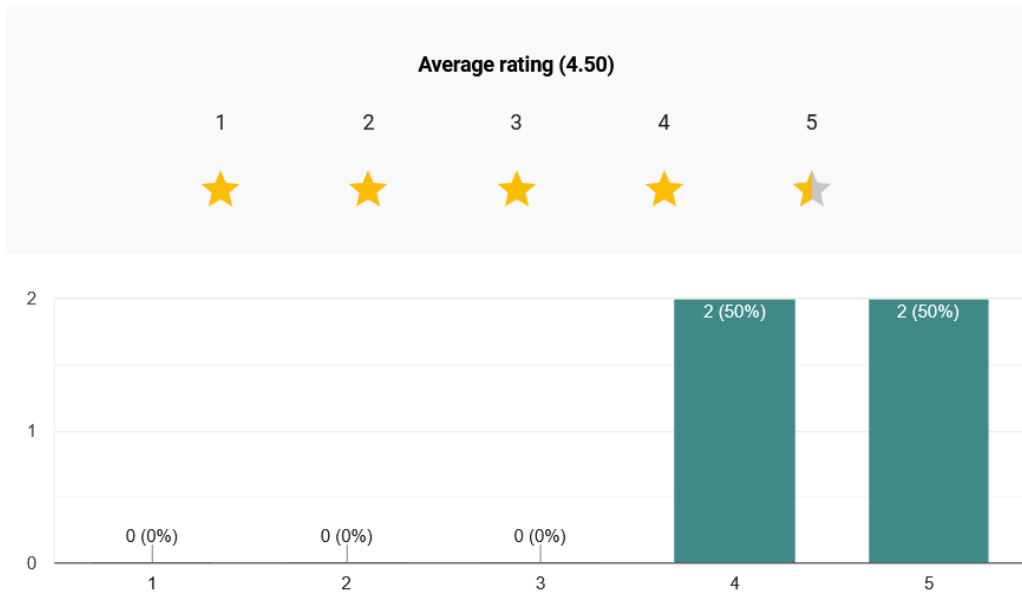
The interface is intuitive and easy to navigate.

4 responses



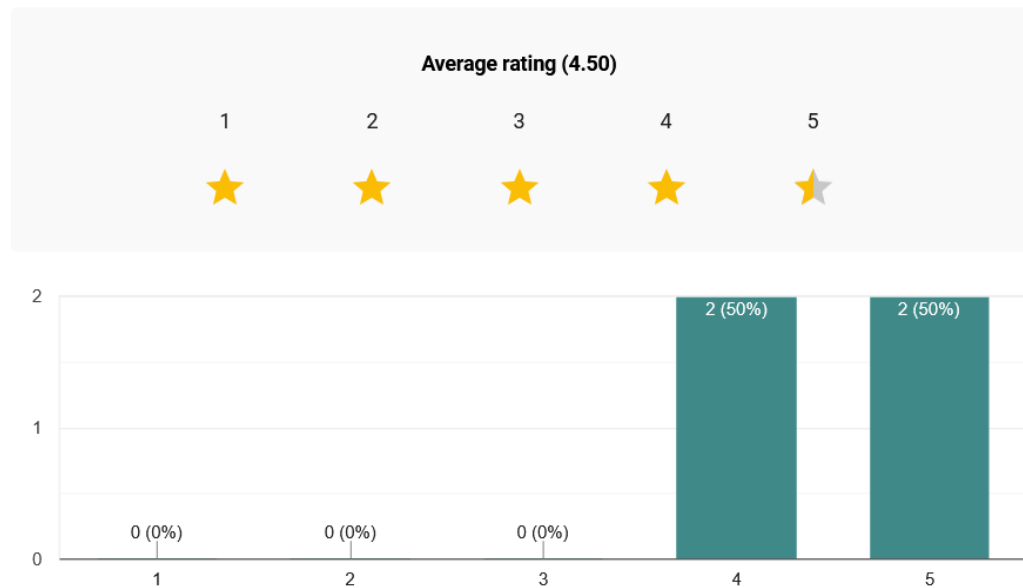
The system effectively integrates detection and education into a cohesive tool.

4 responses



The tool would be useful for improving phishing awareness among non-technical users.

4 responses



Do you have any suggestions for improvement?

3 responses

The idea of integrating the AI-based phishing detection and analysis with an interactive quiz-like system is a great idea. This project would be more meaningful if you could add the non-technical users to use & test your tool and provide a comprehensive survey or questionnaires for them to give feedback because they are the main targeted users for this project. You need to consider the proper mechanisms/procedures about how the non-technical users can do the email phishing detection and analysis using the eml file format as an input (How they can retrieve this file format?). The detection accuracy also can be improved, especially to cater the legitimate email contents that have been considered as threats (high risk score).

Educational training needs to more interactive and rewarding.  
The System can be scaled to verify based only on the content of the email

Though the system performance is promising based on a smaller dataset of phishing email examples, a bigger dataset may enhance the system efficiency further.



## Appendix E.

The following excerpts present the core modules discussed in Chapter 4. Each listing is trimmed to the essential logic; boilerplate imports and error-handling wrappers are omitted for brevity. The complete source code for both the web application and the ML training notebook is submitted as a separate digital archive alongside this report.

**Listing E.1: Feature extraction pipeline (feature\_extractor.py).** This module computes the 22 engineered features described in Section 3.4. The three extractor functions: header, body, and URL are ported directly from the training notebook to guarantee feature parity between training and inference.

```
# app/services/feature_extractor.py (selected excerpts)

URGENCY_WORDS = [
    "verify", "immediately", "urgent", "suspended", "limited", "expire",
    "confirm", "update", "validate", "click here", "act now",
    "account locked", "unusual activity", "security alert",
    "unauthorised", "unauthorized", "compromised", "blocked", "unusual",
    "restricted",
]

CREDENTIAL_WORDS = [
    "password", "username", "login", "sign in", "credentials", "ssn",
    "credit card",
]

FINANCIAL_WORDS = [
    "bank account", "wire transfer", "western union", "inheritance",
    "lottery", "prize",
]

BRAND_DOMAINS = {
    "paypal": "paypal.com", "amazon": "amazon.com",
    "apple": "apple.com", "microsoft": "microsoft.com",
    "google": "google.com", "facebook": "facebook.com",
    "netflix": "netflix.com",
}

SUSPICIOUS_TLDS = {
    "tk", "ml", "ga", "cf", "gq", "xyz", "top", "club", "work",
}

def extract_header_features(row):
    features = {}
    sender = str(row.get("sender", ""))
    body_raw = str(row.get("body", ""))

    sender_domain = ""
    match = re.search(r"@([\w\.-]+)", sender)
    if match:
        sender_domain = match.group(1).lower()

    # Reply-To mismatch
    reply_to_match = re.search(
        r"Reply-To:.*?@([\w\.-]+)", body_raw, re.IGNORECASE
    )
    reply_domain = reply_to_match.group(1).lower() if reply_to_match else ""
    features["sender_reply_mismatch"] = int(
        bool(sender_domain and reply_domain and sender_domain != reply_domain)
    )

    features["has_spf"] = int(bool(re.search(r"spf=pass", body_raw, re.IGNORECASE)))
    features["has_dkim"] = int(bool(re.search(r"dkim=pass", body_raw, re.IGNORECASE)))
    features["received_hops"] = len(re.findall(r"^Received:", body_raw, re.MULTILINE))

    # Display name spoofing
    spoofed = 0
    for brand, legit_domain in BRAND_DOMAINS.items():
        if brand in sender.lower() and legit_domain not in sender_domain:
```

```

        spoofed = 1
        break
    features["display_name_spoofed"] = spoofed
    return features

def extract_body_features(row):
    features = {}
    body = str(row.get("body", ""))
    subject = str(row.get("subject", ""))
    full_text = (subject + " " + body).lower()

    features["urgency_keyword_count"] = sum(
        full_text.count(kw) for kw in URGENCY_WORDS
    )
    features["has_html"] = int("<html" in body.lower() or "<a href" in body.lower())

    html_chars = sum(len(tag) for tag in re.findall(r"<[>]+>", body))
    total_chars = len(body)
    features["html_to_text_ratio"] = (
        round(html_chars / total_chars, 4) if total_chars > 0 else 0
    )

    body_urls = re.findall(r'https?://[^\s<>"\']+', body)
    features["url_count"] = len(body_urls)
    url_chars = sum(len(u) for u in body_urls)
    features["body_url_ratio"] = (
        round(url_chars / total_chars, 4) if total_chars > 0 else 0
    )

    words = re.findall(r'\b[a-zA-Z]+\b', body)
    features["avg_word_length"] = (
        round(float(np.mean([len(w) for w in words])), 2) if words else 0
    )
    features["exclamation_count"] = body.count("!")
    features["credential_request"] = int(any(w in full_text for w in CREDENTIAL_WORDS))
    features["financial_threat"] = int(any(w in full_text for w in FINANCIAL_WORDS))
    return features

def extract_url_features(row):
    features = {
        "url_uses_ip": 0, "url_has_at_symbol": 0, "url_length": 0,
        "url_dot_count": 0, "url_has_https": 0, "url_subdomain_depth": 0,
        "url_suspicious_tld": 0, "external_domain_count": 0,
    }

    body_urls = re.findall(r'https?://[^\s<>"\']+', str(row.get("body", "")))
    if not body_urls:
        return features

    url = body_urls[0] # First URL only (notebook convention)
    features["url_length"] = len(url)
    features["url_has_at_symbol"] = int("@" in url)
    features["url_has_https"] = int(url.startswith("https"))
    features["url_uses_ip"] = int(
        bool(re.search(r'https?://\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}', url))
    )

    extracted = tldextract.extract(url)
    subdomain = extracted.subdomain
    features["url_subdomain_depth"] = len(subdomain.split(".")) if subdomain else 0
    features["url_dot_count"] = url.count(".")
    features["url_suspicious_tld"] = int(extracted.suffix in SUSPICIOUS_TLDS)

    # Unique external domains across ALL URLs
    unique_domains = set()
    for u in body_urls:
        ext = tldextract.extract(u)
        if ext.domain:
            unique_domains.add(f"{ext.domain}.{ext.suffix}")
    features["external_domain_count"] = len(unique_domains)
    return features

def extract_all_engineered_features(parsed_email):
    """Run all three extractors and merge into one flat dict."""
    features = {}
    features.update(extract_header_features(parsed_email))
    features.update(extract_body_features(parsed_email))

```

```

features.update(extract_url_features(parsed_email))
return features

```

**Listing E.2: Explanation engine (explanation.py).** This module implements the rule-based flag generation described in Section 4.3, translating engineered feature values into human-readable threat flags grouped by category and sorted by severity.

```

# app/services/explanation.py (selected excerpts)

def generate_flags(engineered, probability):
    """Build threat flags triggered by extracted features."""
    flags = []

    # — Header flags —
    if engineered.get("display_name_spoofed", 0) == 1:
        flags.append({
            "category": "header", "severity": "high",
            "title": "Display name impersonates a known brand",
            "description": "The sender name mentions a well-known company "
                           "but the actual email domain does not belong to that company.",
            "recommendation": "Never trust a display name alone - always "
                              "check that the domain after '@' matches the company's "
                              "real website.",
        })

    if engineered.get("sender_reply_mismatch", 0) == 1:
        flags.append({
            "category": "header", "severity": "high",
            "title": "Reply-To address does not match sender",
            "description": "The Reply-To header points to a different "
                           "domain than the From address.",
            "recommendation": "Do not reply. Contact the organisation "
                              "through a trusted channel.",
        })

    # — URL flags —
    if engineered.get("url_uses_ip", 0) == 1:
        flags.append({
            "category": "url", "severity": "high",
            "title": "Link uses a raw IP address",
            "description": "At least one link points to a numeric IP "
                           "address instead of a domain name.",
            "recommendation": "Do not click the link.",
        })

    if engineered.get("url_suspicious_tld", 0) == 1:
        flags.append({
            "category": "url", "severity": "medium",
            "title": "Link uses a high-risk top-level domain",
            "description": "One or more links use a TLD frequently "
                           "abused by phishing operators (e.g. .tk, .xyz).",
            "recommendation": "Treat the link with extreme suspicion.",
        })

    # — Content flags —
    urgency = int(engineered.get("urgency_keyword_count", 0) or 0)
    if urgency >= 5:
        flags.append({
            "category": "content",
            "severity": "high" if urgency >= 10 else "medium",
            "title": f"High use of urgency language ({urgency} occurrences)",
            "description": "The email contains many urgency phrases such "
                           "as 'verify immediately' or 'account locked'.",
            "recommendation": "Pause before reacting. Real institutions "
                              "give you time to verify through official channels.",
        })

    if engineered.get("credential_request", 0) == 1:
        flags.append({
            "category": "content", "severity": "high",
            "title": "Email asks for credentials",
            "description": "The email references passwords or login "
                           "details. Legitimate services do not ask for these "
                           "in response to an email.",
            "recommendation": "Never enter your password on a page "

```

```

        "you reached via an email link.",
    })

    if engineered.get("financial_threat", 0) == 1:
        flags.append({
            "category": "content", "severity": "high",
            "title": "Financial scam keywords detected",
            "description": "The email contains language typical of "
                "advance-fee fraud or inheritance scams.",
            "recommendation": "Do not respond or send money.",
        })

    # Sort high → low severity
    severity_rank = {"high": 0, "medium": 1, "low": 2}
    flags.sort(key=lambda f: severity_rank.get(f["severity"], 3))
    return flags

def verdict(probability):
    """Map probability to one of four user-facing verdict bands."""
    if probability >= 0.80: return "High Risk - Likely Phishing"
    if probability >= 0.50: return "Elevated Risk - Suspicious"
    if probability >= 0.20: return "Low Risk - Probably Safe"
    return "Minimal Risk - Likely Legitimate"

def risk_score(probability):
    """Convert P(phishing) to a 0-100 integer score."""
    return int(round(probability * 100))

```

**Listing E.3: ML classifier inference pipeline (ml\_classifier.py).** This module loads the four trained artefacts at startup and exposes the `classify()` function described in Section 4.3, which assembles the 322-feature vector and returns the model's prediction with the engineered feature values needed by the explanation engine.

```

# app/services/ml_classifier.py (selected excerpts)

# Singleton state - populated once by load_model()
_model = None
_tfidf_subject = None
_tfidf_body = None
_feature_columns = None
_subject_tfidf_cols = []
_body_tfidf_cols = []

def load_model(models_dir):
    """Load all four ML artefacts from disk into module-level singletons."""
    global _model, _tfidf_subject, _tfidf_body, _feature_columns
    global _subject_tfidf_cols, _body_tfidf_cols

    _model = joblib.load(os.path.join(models_dir, "phishing_model.pkl"))
    _tfidf_subject = joblib.load(os.path.join(models_dir, "tfidf_subject.pkl"))
    _tfidf_body = joblib.load(os.path.join(models_dir, "tfidf_body.pkl"))
    _feature_columns = list(joblib.load(
        os.path.join(models_dir, "feature_names.pkl")
    ))

    _subject_tfidf_cols = [
        f"subj_{t}" for t in _tfidf_subject.get_feature_names_out()
    ]
    _body_tfidf_cols = [
        f"body_{t}" for t in _tfidf_body.get_feature_names_out()
    ]

    # Sanity check
    expected = getattr(_model, "n_features_in", None)
    if expected and expected != len(_feature_columns):
        raise RuntimeError("Model/feature mismatch - re-export .pkl files.")

def classify(parsed_email):
    """
    Run inference: engineered features + TF-IDF → prediction.

    Returns dict with 'prediction', 'probability', 'engineered',
    and 'feature_vector' keys.
    """

```

```

"""
# Step 1: 22 engineered features
engineered = extract_all_engineered_features(parsed_email)

# Step 2: TF-IDF features (100 subject + 200 body)
feats = dict(engineered)

subject_text = parsed_email.get("subject", "") or ""
body_text = parsed_email.get("body", "") or ""

subj_vec = _tfidf_subject.transform([subject_text]).toarray()[0]
for col, val in zip(_subject_tfidf_cols, subj_vec):
    feats[col] = val

body_vec = _tfidf_body.transform(
    [clean_body_for_tfidf(body_text)]
).toarray()[0]
for col, val in zip(_body_tfidf_cols, body_vec):
    feats[col] = val

# Step 3: reindex into training column order
x_series = pd.Series(feats).reindex(_feature_columns).fillna(0)
x_array = x_series.values.reshape(1, -1)

# Step 4: predict
prediction = int(_model.predict(x_array)[0])
probability = float(_model.predict_proba(x_array)[0][1])

return {
    "prediction": prediction,
    "probability": probability,
    "engineered": engineered,
    "feature_vector": feats,
}

```

**Listing E.4: Representative educational scenario (seed\_scenarios.py).** One of the sixteen scenario objects used to seed the educational platform database. Each scenario contains the simulated email, a list of phishing indicators for the feedback panel, a narrative explanation, and structured learning points.

```
# seed_scenarios.py (one scenario of sixteen)

{
    "title": "PayPal Account Locked - Verify Now",
    "difficulty": "Beginner",
    "is_phishing": True,
    "email_content": """From: security@paypal-support.com
To: customer@example.com
Subject: Your PayPal Account Has Been LOCKED

Dear PayPal Customer,

We have detected unusual activity on your PayPal account.
For your security, your account has been temporarily locked.

To restore access, you must verify your identity within 24 hours:
CLICK HERE TO VERIFY: http://paypal-support.com/verify/login

You will need to confirm:
- Your full name
- Password
- Credit card number and CVV
- Social Security Number

Failure to verify will result in permanent account suspension.

PayPal Security Team
""",
    "indicators": [
        "Lookalike domain: 'paypal-support.com' (1 replacing l)",
        "Requests extremely sensitive data: password, SSN, CVV",
        "Threat language: 'permanent account suspension'",
        "Generic greeting: 'Dear PayPal Customer'",
        "Urgency: '24 hours'",
    ],
    "explanation": (
        "PayPal would NEVER ask for your password, credit card CVV, "
        "or Social Security Number via email. The sender domain "
        "'paypal-support.com' uses a 'l' to impersonate 'paypal' - "
        "this is called typosquatting. The threat of permanent "
        "suspension is designed to panic you into complying."
    ),
    "learning_points": [
        "Legitimate services NEVER ask for passwords via email.",
        "Check URLs carefully - 'l' replacing 'l' is typosquatting.",
        "If unsure, log in directly at paypal.com - never via email links.",
        "No legitimate company needs your SSN for account verification.",
    ],
}
```